



COMSATS University Islamabad, Pakistan

foodO

A project presented to

COMSATS Institute of Information Technology, Islamabad

In partial fulfillment of the requirement for the degree of

Bachelor of Science in Computer Science (2022-2026)

By

Muhammad Abdullah

CUI/SP22-BCS-059/SWL

Muhammad Jawad Saeed

CUI/SP22-BCS-081/SWL

Muhammad Taha Azhar

CUI/SP22-BCS-092/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Muhammad Abdullah

Muhammad Jawad Saeed

Muhammad Taha Azhar

CERTIFICATE OF APPROVAL

It is to certify that the final year project “foodO” was developed by **Muhammad Abdullah (CUI/SP22-BCS-059/SWL)**, **Muhammad Jawad Saeed (CUI/SP22-BCS-081/SWL)** and **Muhammad Taha Azhar (CUI/SP22-BCS-092/SWL)** under the supervision of MS. SAMRA FIRDOUS SABIR and co-supervisor DR. JAVED FERZUND. In their opinion, it is fully adequate in scope and quality for the degree of Bachelor of Science in Computer Science.

Supervisor

Ms. Samra Firdous Sabir
(Department of Computer Science)

Co-Supervisor

Dr. Javed Ferzund
(Department of Computer Science)

Head of Department

Dr. Zafar Roy
(Department of Computer Science)

Executive Summary

foodO is a web-based food delivery ecosystem designed to connect customers, home-based cooks, and independent riders through a unified, efficient, and user-friendly platform. The system addresses the limitations found in traditional delivery services—such as lack of support for home cooks, poor real-time tracking, limited menu management tools, and weak communication between stakeholders—by offering a modern, scalable, and feature-rich solution.

The platform enables customers to browse menus, place orders, track deliveries, and leave reviews through an intuitive and responsive web interface. Home cooks can manage their menus, update order statuses, track earnings, and interact with customers, while riders are provided with a streamlined interface to accept delivery requests, navigate routes, and update delivery progress. The admin panel offers complete oversight of users, orders, payments, and system operations.

Built using Node.js, Express.js, ReactJS, and MongoDB, foodO adopts a modular architecture and follows the Incremental Software Development Methodology, ensuring scalability, maintainability, and continuous improvement. Key innovations of the platform include real-time order and delivery updates via Socket.IO, role-based secure authentication, dynamic menu and order management, and integrated payment and withdrawal systems.

The report presents a comprehensive analysis of the system's requirements, architecture, design models, implementation details, API structure, and testing results. Through this project, modern software engineering principles—such as user-centric design, responsive UI development, real-time communication, and scalable backend architecture—are effectively demonstrated. foodO ultimately provides a robust digital ecosystem that empowers home cooks, supports delivery riders, and delivers a seamless food-ordering experience to customers.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Ms. Samra Firdous Sabir and co-supervisor Dr. Javed Ferzund”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Muhammad Abdullah

Muhammad Jawad Saeed

Muhammad Taha Azhar

Abbreviations

Abbreviation	Full Form
SRS	Software Requirement Specification
SDD	Software Design Document
API	Application Programming Interface
GPS	Global Positioning System
UX	User Experience
UI	User Interface
SDLC	Software Development Life Cycle
JWT	JSON Web Token
CRUD	Create, Read, Update, Delete
HCI	Human-Computer Interaction
VS Code	Visual Studio Code
UUID	Universally Unique Identifier
NoSQL	Non-Structured Query Language
CI	Continuous Integration
ReactJS	React JavaScript (Web Framework)
Node.js	Node JavaScript (Backend Runtime)
MongoDB	Mongo Database (NoSQL DBMS)
REST	Representational State Transfer
Postman	API Testing Tool

Table of Contents

1	INTRODUCTION.....	2
1.1	BRIEF	3
1.2	RELEVANCE TO COURSE MODULES.....	4
1.3	PROJECT BACKGROUND	6
1.3.1	<i>Problem Statement</i>	7
1.3.2	<i>Target Audience</i>	7
1.3.3	<i>Project Objectives</i>	8
1.3.4	<i>Technological Choices</i>	8
1.3.5	<i>System Features</i>	9
1.3.6	<i>Challenges and Innovations</i>	9
1.4	LITERATURE REVIEW	9
1.4.1	<i>Home-Cooked Food Delivery Market Trends</i>	10
1.4.2	<i>Technologies for Building Food Delivery Platforms</i>	10
1.4.3	<i>Backend Development Technologies</i>	10
1.4.4	<i>Web Application Development</i>	11
1.4.5	<i>Real-Time Order Status Tracking and Notifications</i>	11
1.4.6	<i>Challenges in Food Delivery Ecosystem</i>	11
1.4.7	<i>Conclusion from Literature Review</i>	12
1.5	ANALYSIS FROM LITERATURE REVIEW	12
1.5.1	<i>Real-Time Order Tracking and Updates</i>	12
1.5.2	<i>Scalable and Secure Backend Architecture</i>	12
1.5.3	<i>Responsive Web Design and Frontend Development</i>	13
1.5.4	<i>User Experience (UX) and User Interface (UI) Design</i>	13
1.5.5	<i>Real-Time Data Synchronization</i>	13
1.5.6	<i>Conclusion</i>	14
1.6	METHODOLOGY AND SOFTWARE LIFECYCLE FOR THIS PROJECT.....	14
1.6.1	<i>Rationale behind Selected Methodology</i>	14
2	PROBLEM DEFINITION.....	19
2.1	PROBLEM STATEMENT.....	20
2.2	DELIVERABLES AND DEVELOPMENT REQUIREMENTS	21
2.2.1	<i>Key Deliverables</i>	21
2.2.2	<i>Development Requirements</i>	22
2.3	CURRENT SYSTEM.....	23
2.3.1	<i>Lack of Integration in Existing Food Delivery Systems</i>	23
2.3.2	<i>User Experience and Quality Challenges</i>	24
2.3.3	<i>Manual Menu and Order Management</i>	24
3	REQUIREMENT ANALYSIS	26
3.1	USE CASES DIAGRAM(S)	26
3.2	DETAILED USE CASE	30
3.3	FUNCTIONAL REQUIREMENTS	42
3.3.1	<i>Customer Module</i>	42
3.3.2	<i>Cook Module</i>	42
3.3.3	<i>Rider Module</i>	42
3.3.4	<i>Admin Module</i>	43
3.4	NON-FUNCTIONAL REQUIREMENTS.....	43
4	DESIGN AND ARCHITECTURE	46
4.1	SYSTEM ARCHITECTURE	46

4.2	DATA REPRESENTATION [DIAGRAM + DESCRIPTION]	51
4.3	PROCESS FLOW/REPRESENTATION	57
4.4	DESIGN MODELS	61
4.4.1	<i>Sequence Diagram</i>	61
4.4.2	<i>Class Diagram</i>	66
5	IMPLEMENTATION	74
5.1	BACKEND IMPLEMENTATION	74
5.2	FRONTEND IMPLEMENTATION (REACTJS)	75
5.3	DATABASE IMPLEMENTATION (MONGODB)	75
5.4	API INTEGRATION	76
5.5	USER INTERFACE	76
5.5.1	<i>Web Interface (ReactJS)</i>	76
6	API REFERENCE	83
6.1	AUTHENTICATION	83
6.2	PROFILE MANAGEMENT	83
6.3	MENU ITEMS MANAGEMENT	84
6.4	COOK DISCOVERY	85
6.5	ORDERS & CHECKOUT	85
6.6	REVIEWS & RATINGS	86
6.7	MEDIA HANDLING	86
6.8	REAL-TIME FEATURES (SOCKET.IO)	86
6.9	CUSTOMER EVENTS	86
6.10	COOK & RIDER EVENTS	87
7	TESTING AND EVALUATION	89
7.1	MANUAL TESTING	89
7.1.1	<i>System testing</i>	89
7.1.2	<i>Unit Testing</i>	89
7.1.3	<i>Functional Testing</i>	90
7.1.4	<i>Integration Testing</i>	91
7.2	AUTOMATED TESTING	92
8	CONCLUSION AND FUTURE WORK	94
8.1	CONCLUSION	94
8.2	FUTURE WORK	94
	REFERENCES	96

List of Figures

Figure 3. 1 Use Case	26
Figure 4. 1 Architecture Diagram	46
Figure 4. 2 ER Diagram	52
Figure 4. 3 Process flow	58
Figure 4. 4 Sequence Diagram	62
Figure 4. 5 Class Diagram	66
Figure 5. 1 Home Page	77
Figure 5. 2 MenuItem Page	78
Figure 5. 3 Cart Page	79
Figure 5. 4 Cook Interface	80
Figure 5. 5 Rider Interface.....	81

List of Tables

Table 3. 1 Feature Category for use case	30
Table 3. 2 Login use case.....	30
Table 3. 3 Register use case	31
Table 3. 4 Browse Menu use case	32
Table 3. 5 Place Order use case	32
Table 3. 6 Enter Delivery Details use case	33
Table 3. 7 Leave Review use case	34
Table 3. 8 Track Order use case	34
Table 3. 9 Manage Menu use case.....	35
Table 3. 10 View Orders use case	36
Table 3. 11 View Reviews use case	36
Table 3. 12 Update Order Status use case	37
Table 3. 13 View Earning use case	37
Table 3. 14 Create Coupon Codes use case	38
Table 3. 15 Withdraw Money use case.....	38
Table 3. 16 Accept Delivery Request use case	39
Table 3. 17 Update Delivery Status use case	39
Table 3. 18 Manage Users use case	40
Table 3. 19 Manage Kitchen use case	40
Table 3. 20 Withdraw Request use case	41
Table 4. 1 Benefits of Architecture Diagram.....	51
Table 4. 2 Relationships Summary	55
Table 4. 3 Class Diagram Relationship	71
Table 6. 1 Customer Authentication Routes.....	83
Table 6. 2 Cook & Rider Authentication Routes	83
Table 6. 3 Customer Profile Management Routes.....	83
Table 6. 4 Cook & Rider Profile Management Routes	84
Table 6. 5 Menu Items (Cook).....	84
Table 6. 6 Cook Discovery (Customer)	85
Table 6. 7 Customer Orders & Checkout	85
Table 6. 8 Cook & Rider Order Management.....	85
Table 6. 9 Reviews & Ratings	86
Table 6. 10 Media Handling	86
Table 6. 11 Namespaces	86
Table 6. 12 Customer Events	86
Table 6. 13 Cook & Rider Events	87
Table 7. 1 Unit Test 1	90
Table 7. 2 Unit Test 2	90
Table 7. 3 Functional Test	91
Table 7. 4 Integration Test	91
Table 7. 5 Tool Used.....	92

Chapter 1

Introduction

1 Introduction

The rise of digital technologies has transformed the way people browse, order, and receive food, leading to a steady shift from traditional dining toward online food delivery systems. However, despite the growth of major delivery platforms, most are designed for commercial restaurants and large food chains, leaving home-based cooks, small vendors, and independent delivery riders without sufficient support. Customers who prefer fresh, hygienic, and home-cooked meals often struggle to find home chefs they can trust, while riders seeking flexible income face unstructured workflows and limited digital tools. These gaps highlight the need for a system specifically designed to support community-driven food services rather than solely focusing on commercial businesses.

To bridge these gaps, foodO has been developed as a comprehensive web-based ecosystem connecting customers, cooks, and riders on a single platform. Customers can browse menus, place orders, view preparation progress, and track deliveries in real time through a responsive web interface. Home cooks gain a structured environment where they can upload food items, manage menus, process orders efficiently, and track earnings. Riders benefit from a clear delivery workflow that enables them to accept tasks, follow optimized delivery routes, and update delivery statuses with ease. An admin panel ensures smooth platform governance by supervising user registrations, handling potential issues, and maintaining system integrity.

The platform ensures seamless coordination among all users through real-time communication and instant status updates. As soon as a customer places an order, the cook receives a notification and can update preparation stages, which are immediately reflected on the customer's end. Once the meal is ready, the system notifies the rider, who proceeds with pickup and delivery while continuously updating the order's status. The platform's responsive design ensures accessibility across laptops, desktops, and mobile browsers without requiring a standalone mobile application, making the system flexible and easy to use for all user roles.

foodO follows modern software engineering principles that emphasize scalability, modularity, and maintainability. Its architecture clearly separates backend logic, frontend interfaces, and database management, enabling easier integration of future enhancements such as smart recommendations, advanced analytics, or automated delivery optimization. By combining real-time communication features with a structured multi-role workflow, foodO offers a reliable, efficient, and user-friendly solution to the challenges present in home-cooked food delivery systems. This report provides a detailed overview of the project's motivation, technological foundation, design methodology,

development process, and evaluation, forming the basis for the technical discussions presented in subsequent chapters.

1.1 Brief

foodO is a comprehensive web-based platform designed to simplify and digitalize the process of ordering home-cooked meals. The system allows customers to browse available cooks, explore menus, place orders, make payments, and track their deliveries in real-time. Simultaneously, cooks can manage their food listings, track orders, communicate with customers, and handle earnings directly from the system.

Unlike many existing delivery systems, foodO focuses exclusively on homemade food services, enabling small-scale cooks and home chefs to reach a wider audience.

Customer Interfaces

Developed using ReactJS, the customer panel enables users to register, log in, browse home-cooked food from verified cooks, place and track orders, make secure payments, and rate or review meals. Customers can also view order history and communicate with cooks or riders for delivery updates

Cook Interface

Home-based cooks can register on the platform to sell their meals. Through their dashboard, cooks can manage menus, set prices, receive orders, update order status (e.g., preparing, ready for pickup), view transaction history, and withdraw earnings securely.

Rider Interface

The rider module is designed for individuals who deliver food orders and earn income per delivery. Riders can view available delivery requests, accept orders, update delivery progress (e.g., picked up, on the way, delivered), and manage their delivery history and earnings.

Admin Panel

The admin panel provides system-wide oversight. Administrators can manage user registrations (customers, cooks, riders), approve new cooks, monitor active orders, handle payment disputes, generate reports, and ensure platform reliability and security.

Key features of the foodO platform include

- **Secure Authentication & Authorization**

Separate role-based access for customers, cooks, riders, and admin.

- **Dynamic Menu Management**

Cooks can easily create and update food listings.

- **Order Placement & Tracking**

Customers can place and monitor orders in real-time.

- **Rider Delivery Management**

Riders can accept, deliver, and update order statuses.

- **Online Payment & Withdrawals**

Integrated payment gateways for easy transactions.

- **Review & Rating System**

Customers can rate meals and delivery experiences.

- **Admin Dashboard**

Provides complete system monitoring and user management.

- **Real-Time Notifications**

Instant updates for order acceptance, preparation, and delivery using Socket.IO.

The development of foodO follows modern software engineering principles, focusing on modularity, scalability, and a user-centered design. The system is built to provide a smooth and reliable experience for customers, cooks, and riders by ensuring clear workflows, real-time updates, and secure interactions. Through its structured design and efficient implementation, foodO delivers a dependable platform for managing home-cooked food services and online deliveries.

1.2 Relevance to Course Modules

The development of **foodO** directly applies concepts learned across multiple software engineering and computer science course modules. The project serves as a practical implementation of theoretical knowledge, covering areas such as web technologies, database systems, system design, software engineering methodologies, and human–computer interaction. The following modules are particularly relevant:

Web Technologies

Web technologies refer to the set of tools, languages, and frameworks used to develop and deploy web-based applications. They include both frontend technologies, such as HTML for structuring content, CSS for styling, and JavaScript for adding interactivity, along with modern libraries and frameworks like React.js that enable dynamic and responsive user interfaces. On the backend, technologies like Node.js and Express.js handle server-side logic, data processing, and communication with the database. For data storage, systems often use relational databases like MySQL or NoSQL solutions such as MongoDB, which provide flexibility and scalability. Additionally, web applications rely on APIs for integration with third-party services, such as Stripe for secure payment processing and Google Maps for geolocation. Supporting tools like Git for version control and cloud platforms for deployment ensure efficient development and maintenance. Together, these technologies create robust, scalable, and user-friendly web applications that meet modern standards.

Database Systems

foodO uses a NoSQL database to efficiently store and manage data related to users, menu items, orders, payments, reviews, and delivery details. The implementation demonstrates a strong understanding of essential database concepts, including data modeling, CRUD operations, and schema design for multiple user roles such as customers, cooks, riders, and admins. The system also applies query optimization techniques to ensure fast and reliable performance as data volume grows. Overall, the database structure and operations directly reflect principles taught in database management and information systems courses.

Software Engineering Methodologies

The project uses the Incremental Model, allowing foodO to be developed in structured phases such as authentication, menu management, order processing, and delivery tracking. This approach supports continuous integration, improved maintainability, and early testing all central topics in software engineering and SDLC courses.

Human-Computer Interaction (HCI)

Human-Computer Interaction (HCI) principles are applied throughout foodO to create a clean, intuitive, and responsive user interface for all user roles. The design focuses on easy navigation, clear visual hierarchy, responsive layouts, and accessibility, ensuring that users can interact with

the system efficiently and comfortably. These UI/UX practices reflect core concepts from HCI coursework, where user-centered design and usability are key priorities.

System Design and Architecture

foodO incorporates various system design and architecture diagrams, including use case diagrams, sequence diagrams, class diagrams, architecture diagrams, and ER models. These diagrams demonstrate key concepts from system design, UML modeling, and software architecture courses, helping to illustrate how complex systems are structured, interact, and function. This documentation reinforces a strong understanding of designing and representing large-scale software systems in a clear and organized manner.

Software Project Management

The project incorporates essential project management practices such as requirement gathering, planning increments, task scheduling, documentation, version control, and testing. This reflects the principles taught in project management courses, including teamwork, milestone tracking, and structured documentation.

Through the development of foodO, a wide range of academic concepts have been applied, demonstrating the interdisciplinary nature of modern software engineering. The project required solving both frontend and backend challenges while designing an intuitive, user-focused experience for customers, cooks, riders, and administrators. By integrating concepts from web technologies, database systems, HCI, software engineering methodologies, and system design, the development of foodO reflects the comprehensive learning and practical skills gained throughout the course.

1.3 Project Background

The foodO project was developed in response to the rising demand for convenient, reliable, and affordable home-cooked food delivery services. While many online food delivery platforms exist, they primarily focus on restaurant-based businesses, leaving a growing market of home-based cooks underserved. Customers who prefer homemade meals often struggle to find trusted home chefs, while cooks lack a centralized platform to showcase their food, manage orders, and reach a broader audience.

Similarly, young individuals looking for flexible income opportunities through food delivery often lack access to reliable delivery platforms. Existing systems provide limited support for independent riders, making it difficult for them to earn consistently.

foodO aims to bridge these gaps by offering a unified platform that connects customers, home cooks, and delivery riders, enabling seamless order placement, menu management, delivery tracking, and secure transactions—all through a web-based application.

1.3.1 Problem Statement

Existing food delivery platforms often fail to support the needs of home-based cooks, independent riders, and customers seeking home-cooked meals. Many systems lack proper tools for cooks to manage their menus, update order statuses, and track earnings, while customers frequently experience inconsistent interfaces, unclear order updates, and limited communication with service providers. Riders also face challenges due to insufficient features for accepting delivery requests, updating delivery status, and managing payouts.

From a technical perspective, current platforms struggle with scalability, real-time updates, and smooth communication between multiple user roles such as customers, cooks, and riders.

The foodO project was therefore initiated to create a more flexible, scalable, and user-centric platform that addresses these shortcomings. Its primary goal is to provide a comprehensive system where customers can easily order home-cooked meals, cooks can efficiently manage their operations, and riders can seamlessly handle deliveries—ensuring a smooth experience from order placement to final delivery.

1.3.2 Target Audience

foodO serves the following key user groups:

- **Customers**

Individuals searching for freshly cooked, homemade meals delivered to their doorstep. They can browse menus, place orders, track deliveries, and provide feedback.

- **Home-Based Cooks (Vendors)**

Independent cooks who prepare and sell homemade food. They use foodO to manage menus, receive and process orders, update order statuses, and handle earnings or withdrawals.

- **Riders (Delivery Partners)**

Independent riders who pick up orders from cooks and deliver them to customers. They rely on the platform to accept delivery tasks, track routes, update delivery status, and manage their earnings.

In addition, an **Admin** role oversees overall platform operations, including user management, cook verification, rider monitoring, and system maintenance.

1.3.3 Project Objectives

The primary objective of the foodO project is to create a unified and user-friendly platform where customers, home-based cooks, and riders can interact seamlessly. The system aims to streamline the process of ordering home-cooked meals by offering customers an intuitive interface for browsing menus, placing orders, and tracking deliveries. For cooks, the platform provides efficient tools for managing menus, handling orders, updating order statuses, and monitoring earnings. Riders are supported with real-time delivery assignments, route tracking, and delivery status updates to improve efficiency and transparency. On the system side, foodO is designed with scalability, reliability, and real-time communication in mind, ensuring smooth operations even as the number of users grows. Overall, the project aims to enhance the entire home-food delivery ecosystem by improving accessibility, communication, and operational efficiency for all stakeholders.

1.3.4 Technological Choices

To meet the project's objectives, the following technologies were selected:

- **Node.js**
Used as the backend runtime environment due to its speed, scalability, and ability to handle concurrent user requests efficiently.
- **Express.js**
A lightweight backend framework that simplifies routing, middleware handling, and API development, making the system modular and easy to maintain.
- **MongoDB**
A NoSQL database chosen for its flexibility and ability to store dynamic data structures such as users, orders, menus, reviews, payments, and delivery details.
- **React.js**
Used for building the web-based interfaces for customers, cooks, riders, and admin. It enables fast rendering, reusable components, and responsive UI design.
- **Socket.IO**
Integrated to support real-time features, including instant order updates, live delivery tracking, notifications, and communication between system roles.

1.3.5 System Features

The foodO platform provides a complete digital ecosystem for customers, home-based cooks, and riders, ensuring smooth interactions across all stages of food ordering and delivery. Customers can browse available meals, place orders, make payments, track their delivery in real time, and share feedback through a built-in review system. Cooks are provided with tools to manage their menus, update order statuses, monitor sales, and handle withdrawal requests efficiently. Riders receive delivery requests, update delivery progress, and complete orders through their dedicated interface. The system includes secure authentication for all roles, real-time notifications for order updates, and an admin dashboard for platform oversight, user management, and system monitoring.

1.3.6 Challenges and Innovations

Developing a platform like foodO required addressing several technical and operational challenges to support customers, home-based cooks, and independent riders effectively.

- **Real-Time Order Processing**

Ensuring that orders placed by customers are instantly visible to cooks and riders so that food preparation and delivery can begin without delays.

- **Scalability**

Designing the backend to handle a growing number of users, cooks, riders, orders, and menu updates while maintaining fast and reliable performance.

- **Multi-Role Interaction**

Coordinating seamless communication between customers, cooks, riders, and admins to ensure smooth order flow, delivery tracking, and system-wide transparency.

The project also introduced innovative solutions such as dynamic menu management for home-based cooks, secure earnings and withdrawal handling for riders and cooks, and real-time delivery notifications to enhance user experience and system efficiency.

1.4 Literature Review

Research on modern food delivery platforms shows a strong focus on creating efficient, user-friendly, and real-time web systems that connect customers with food providers and delivery personnel. Studies emphasize the need for intuitive interfaces, secure authentication, scalable backend architectures, and flexible databases to handle dynamic data such as menus, orders, and payments. Literature on real-time communication also highlights the importance of instant updates for order tracking and delivery coordination. Additionally, Human-Computer Interaction (HCI)

research supports using clear visual design and responsive layouts to enhance usability. These findings align directly with foodO's design, which incorporates a web-based multi-role system, real-time order updates, and a scalable NoSQL database structure to meet the evolving needs of online food services.

1.4.1 Home-Cooked Food Delivery Market Trends

The online food delivery (OFD) market has demonstrated robust global growth, driven primarily by increasing urbanization, high smartphone penetration, and a growing consumer preference for convenience [1]. The shift in consumer behavior, influenced by busy lifestyles and time constraints, has cemented digital ordering as a standard service model [2]. While major aggregators dominate the restaurant delivery segment, the market exhibits a clear demand for specialized, niche services. The foodO platform is strategically positioned to address this gap by focusing specifically on home-cooked meals, offering an alternative that prioritizes authenticity, freshness, and community support over mass-produced food [3]. This focus aligns with the trend of platforms seeking to deliver personalized and high-quality meal options to consumers [1].

1.4.2 Technologies for Building Food Delivery Platforms

To effectively manage the complexity of a multi-sided marketplace—connecting Customers, Cooks, and Riders—a robust and scalable technology stack is imperative. For the foodO platform, the MERN (MongoDB, Express.js, React JS, Node.js) stack was selected. This stack is highly regarded in the industry for developing modern, scalable, and high-performance web applications [4]. The primary benefit of the MERN stack is the uniformity of the JavaScript language across the entire application—from front-end to back-end—which simplifies development, enhances productivity, and provides a cohesive ecosystem [5].

1.4.3 Backend Development Technologies

The backend of the foodO system utilizes Node.js coupled with the Express.js framework. Node.js is an event-driven, non-blocking I/O runtime environment, making it exceptionally suitable for handling the high volume of concurrent connections essential for a real-time ordering and tracking system [5]. This architecture ensures low latency and high throughput, which are critical during peak ordering times. The database layer, MongoDB, is a NoSQL database that offers flexibility and scalability, allowing the system to easily store and manage dynamic data types, such as variable menu listings, evolving user profiles, and constantly changing order statuses [4].

1.4.4 Web Application Development

React JS serves as the foundation for the client-side user interfaces. Its component-based architecture facilitates the rapid development and modularity required for building the four distinct, role-based dashboards (Customer, Cook, Rider, and Admin) that define foodO [5]. React's use of the Virtual DOM ensures that the user interface is rendered efficiently, providing a smooth and responsive experience across various devices and screen sizes (a non-functional requirement for usability) [4].

1.4.5 Real-Time Order Status Tracking and Notifications

The ability to provide instantaneous updates for order status and delivery location is a key functional requirement for customer satisfaction in food delivery [6]. Traditional web communication protocols (like HTTP polling) are inefficient for this purpose, as they consume excessive bandwidth and impose heavy loads on the server [7]. To achieve true real-time functionality, foodO integrates WebSockets (via libraries like Socket.IO). This technology establishes a persistent, full-duplex communication channel between the client and server, allowing for immediate, low-latency data exchange. This feature is vital for live order tracking, instant notifications of order acceptance, preparation completion, and rider arrival [5], [7].

1.4.6 Challenges in Food Delivery Ecosystem

The operational landscape of food delivery presents several persistent challenges that the foodO architecture is specifically engineered to overcome. Foremost among these are the issues of logistical complexity and delivery delays, where ensuring timely order fulfillment and optimizing delivery routes remain critical operational hurdles [8]. Furthermore, given the decentralized nature of the platform involving multiple independent home cooks, quality control and consistency emerge as a significant challenge; maintaining a uniform standard of food quality across this network directly impacts customer satisfaction and retention [8]. In terms of platform integrity, security and trust are paramount, necessitating robust measures for protecting sensitive user data, ensuring the seamless and secure integration of payment gateways, and establishing transparent, fair fee structures necessary for building user confidence [9]. Finally, effective Rider Management is crucial for operational efficiency, requiring a system capable of handling the gig economy workforce, including the fair and clear distribution of delivery tasks, and maintaining open communication channels to coordinate the last mile of delivery [9].

1.4.7 Conclusion from Literature Review

The review confirms a sustained market demand for convenient and high-quality food delivery services, validating the market opportunity for a specialized platform like foodO. The chosen MERN stack, coupled with the Agile-Iterative Methodology (as per project documentation), is technically sound and capable of handling the inherent complexities and real-time demands of the platform. By focusing on role-based access, secure payments, and leveraging WebSockets for real-time tracking, foodO is well-equipped to provide a competitive, scalable, and reliable service in the home-cooked food sector.

1.5 Analysis from Literature Review

The analysis confirms a strong market demand for specialized, high-quality food delivery, validating the foodO platform's focus on home-cooked meals. This requirement for a scalable, high-performance system justifies the choice of the MERN stack (Node.js/React JS/MongoDB) due to its efficiency and unified JavaScript environment. Critically, to address the need for transparency and reliability in delivery logistics, the architecture is designed to integrate WebSockets (Socket.IO) for essential real-time order tracking and notifications. This strategic technical foundation ensures that foodO is equipped to mitigate industry challenges, including logistics, quality control via a rating system, and security via Role-Based Access Control, making the proposed solution both market-relevant and technically sound.

1.5.1 Real-Time Order Tracking and Updates

The literature emphasizes that customer satisfaction in the online food delivery (OFD) market is highly dependent on transparency and timely information [8], [9]. Consequently, real-time features, such as order status updates and delivery tracking, are a critical non-functional requirement [5]. Traditional request-response HTTP protocols are inefficient for this continuous data flow [6]. The analysis confirms that the use of WebSockets (e.g., via Socket.IO) is the superior architectural choice, as it establishes a persistent, full-duplex communication channel between the client and server. This low-latency connection is necessary to instantly notify all parties (Customer, Cook, and Rider) of critical events, ensuring seamless status synchronization and tracking [6], [7].

1.5.2 Scalable and Secure Backend Architecture

The market for online food delivery is rapidly expanding [1], [3], requiring the foodO platform to be highly scalable and capable of managing a high volume of concurrent users [4]. The selection of Node.js and Express.js is validated because their event-driven, non-blocking I/O model is uniquely suited for high-throughput, real-time applications, efficiently managing concurrent

requests without performance degradation [4], [5]. Furthermore, the need to protect sensitive user and financial data, a major challenge in the ecosystem, necessitates a strong security framework [9]. The backend is designed to handle secure authentication, payment gateway integration, and transaction security, a capability supported by the robustness of the chosen Node.js/Express.js technology stack [5].

1.5.3 Responsive Web Design and Frontend Development

Due to the high usage of smartphones for ordering and the necessity of managing orders from diverse devices (desktop for Admin/Cook, mobile for Rider/Customer) [2], Responsive Web Design (RWD) is a fundamental requirement [5]. React JS is the ideal frontend technology, offering a component-based architecture that facilitates rapid development and promotes the reuse of code across the four distinct, role-based interfaces (Customer, Cook, Rider, and Admin) [4], [5]. This approach ensures that the application maintains a consistent and intuitive experience regardless of the access device.

1.5.4 User Experience (UX) and User Interface (UI) Design

Effective UX and UI design are crucial for minimizing customer effort and boosting satisfaction [2]. The literature highlights that an intuitive interface simplifies the process of browsing menus, placing orders, and tracking deliveries [8]. The component-based nature of React JS allows for the modular construction of intuitive, user-friendly interfaces tailored to the specific functions of each role (e.g., dynamic menu management for the Cook vs. quick order placement for the Customer) [5]. This design choice directly addresses the need for a simplified and accessible ordering experience for all users [2].

1.5.5 Real-Time Data Synchronization

Beyond basic tracking, the complexity of managing orders across three different stakeholders (Cook, Rider, Customer) requires immediate and flawless data synchronization. The chosen technology stack, particularly the combination of MongoDB (for flexible, dynamic data storage) and Node.js coupled with WebSockets, is the primary mechanism for achieving this. This setup allows for instantaneous updates—such as menu changes, order status transitions, and fund withdrawal processing—to be broadcast and reflected across all relevant dashboards simultaneously, mitigating the risks associated with data latency and ensuring operational consistency [4], [6].

1.5.6 Conclusion

The analysis of the food delivery literature firmly validates the foodO project's market positioning and its technical blueprint. The market is ripe for a specialized home-cooked food platform [1], [3]. The technical choices—the MERN stack and WebSockets—are directly supported by academic and industry literature as the most effective solution for building a secure, highly scalable, and real-time multi-stakeholder application [4], [5]. By implementing these technologies, foodO is well-equipped to overcome the operational challenges identified in the ecosystem and deliver a reliable, user-centric service.

1.6 Methodology and Software Lifecycle for This Project

The development of the **foodO** platform followed an Agile-Iterative Software Development Lifecycle (SDLC), which allowed the team to deliver the system in incremental stages while accommodating evolving requirements. The process began with requirement analysis, where functional and non-functional specifications were documented in the SRS. Next, system design was carried out, including architecture diagrams, class diagrams, and data models, as detailed in the SDD. Implementation was performed using the MERN stack (MongoDB, Express.js, React, Node.js), ensuring a consistent JavaScript environment across all layers. Each sprint focused on building core modules such as authentication, menu management, and order tracking, followed by rigorous unit and integration testing to validate functionality. Deployment was handled through a cloud-based environment to ensure scalability and accessibility, while continuous integration and delivery pipelines streamlined updates. Finally, maintenance activities were planned to address bug fixes, feature enhancements, and performance improvements. This iterative approach ensured flexibility, early feedback, and reduced risk throughout the project lifecycle.

1.6.1 Rationale behind Selected Methodology

The Agile-Iterative methodology was chosen because it offers adaptability and responsiveness to changing requirements, which is critical for a dynamic platform like foodO. Unlike traditional models, Agile enables incremental delivery, allowing stakeholders to review and validate features such as user registration, payment integration, and real-time tracking at the end of each sprint. This approach minimizes risks by incorporating continuous testing and feedback loops, ensuring that issues are identified early rather than at the final stage. Additionally, Agile promotes collaboration and transparency among team members, which was essential for coordinating multiple roles and modules. The modular design of foodO aligns perfectly with Agile principles, making it easier to scale and integrate advanced features like WebSocket-based real-time communication and secure

payment gateways. Overall, Agile provided the flexibility and efficiency needed to deliver a robust, user-centric solution within the given timeline.

1.6.1.1 Flexibility and Adaptability

The Incremental Model provides a high level of flexibility, making it well-suited for foodO, where new requirements can emerge as the platform grows and user expectations evolve. Since foodO involves multiple user roles—customers, cooks, riders, and admins—each with unique workflows, the system benefits greatly from being developed in stages. By delivering the system in small increments, the development team can refine features after each build and adjust to feedback from real users. This adaptability is especially important in the home-cooked food delivery domain, where user preferences, service patterns, and operational needs can change quickly. The iterative nature of the Incremental Model enables continuous improvement without requiring major structural changes to the entire system.

1.6.1.2 Early Deliverables and Feedback

The Incremental Model allows early versions of the foodO platform to be released for real-world use, which helps the development team gather meaningful feedback during the early stages of development. Initial increments may include fundamental features such as customer registration, menu browsing, and order placement, while later increments introduce more complex functionalities like delivery tracking, earning withdrawals, and real-time notifications. Early user interaction—from customers browsing dishes, cooks uploading menus, or riders accepting delivery tasks—provides practical insights that help refine the system. This approach ensures that each subsequent increment aligns more closely with the actual needs and expectations of the platform’s users, resulting in a more polished and user-centered final system.

1.6.1.3 Risk Management

Risk identification and management are greatly improved through the Incremental Model because each increment is developed and tested independently. For a system like foodO, which includes real-time order updates, multiple user roles, payment handling, and communication flows, risks can arise in areas such as data accuracy, system scalability, or delivery coordination. By developing the platform in smaller sections, potential issues—such as performance limitations in the backend, conflicts in order state management, or delays in real-time updates—can be detected early. Addressing these risks before moving to more complex modules helps prevent major failures later in the development process. Continuous evaluation at the end of each increment ensures that problems do not accumulate and disrupt the overall system.

1.6.1.4 Continuous Improvement

Developing foodO in increments supports ongoing refinement and quality enhancement. Each increment builds upon the functionality introduced in previous stages, allowing the development team to evolve system features based on real usage and performance observations. As core features like order placement, delivery management, and earnings tracking are deployed, they can be gradually enhanced with additional functionalities such as improved UI/UX flows, advanced filtering options, or optimized route-updates for riders. This iterative process ensures that the platform stays aligned with the needs of its users while integrating new improvements efficiently. Continuous integration and testing at the end of each increment also contribute to a more stable and reliable system.

1.6.1.5 Clear Milestones and Progress Tracking

The Incremental Model establishes clear and measurable milestones, simplifying project tracking and management. Each increment represents a functional component of foodO—such as menu management, delivery workflow, or admin monitoring—which can be reviewed, tested, and validated independently. This clarity enables both the development team and stakeholders to monitor progress consistently and assess whether the system is moving toward its intended goals. Defined milestones help maintain focus, ensure timely delivery of features, and provide transparency throughout the development cycle. They also make it easier to allocate tasks, manage timelines, and communicate development status with all stakeholders involved.

1.6.1.6 Support for Ongoing Development

foodO is designed to evolve continuously as new needs emerge from customers, home cooks, and riders. The Incremental Model supports this long-term evolution by allowing new modules to be added without disrupting existing functionality. After the initial launch of the platform, future increments can introduce enhancements such as promotional systems, cook performance analytics, rider route optimization, loyalty programs, or AI-driven order recommendations. Because each increment builds independently yet integrates seamlessly into the main system, foodO can remain competitive and adapt to future technological trends or service requirements without requiring major redevelopment efforts.

1.6.1.7 Conclusion

The Incremental Model proves to be an ideal development methodology for foodO, offering flexibility, early user feedback, effective risk mitigation, clear progress tracking, and strong support for long-term system evolution. By dividing the system into manageable increments, the

development process becomes more controlled, adaptive, and user-oriented. This approach ensures that foodO remains scalable, reliable, and capable of meeting the diverse needs of customers, home cooks, riders, and administrators. Ultimately, the Incremental Model enables foodO to grow in a structured and stable manner, evolving alongside user expectations and industry advancements.

Chapter 2

Problem Definition

2 Problem Definition

The food delivery industry is growing rapidly, yet existing platforms primarily cater to restaurant-based services, leaving a gap for customers who prefer authentic, home-cooked meals. Current systems often lack personalization, seamless integration, and efficient backend processes, resulting in challenges such as limited menu diversity, high service fees, and inconsistent delivery experiences. Home chefs struggle to reach customers due to the absence of dedicated platforms that support menu management, secure payments, and real-time order tracking. Additionally, operational issues such as scalability, performance optimization, and real-time communication remain critical for ensuring smooth transactions and customer satisfaction.

To address these challenges, the foodO platform is designed as a comprehensive web-based solution that connects customers with verified home cooks and delivery riders. It offers role-based dashboards, intuitive user interfaces, and robust backend services to streamline the ordering and delivery process. However, the development of such a platform introduces several technical and operational challenges, which can be categorized as follows:

These challenges are reflected in several key areas

- **User Experience (UX) Issues**

The need for an intuitive, responsive, and user-friendly interface for customers, cooks, and riders to ensure smooth navigation and interaction.

- **Seamless Integration**

Difficulty in integrating multiple modules—frontend interfaces, backend services, and databases—to maintain consistent communication and data flow.

- **Scalability**

Building an infrastructure capable of handling increasing traffic and expanding user and cook bases as the platform grows.

- **Performance and Efficiency**

Ensuring fast and reliable performance for time-sensitive operations such as order placement, payment processing, and delivery tracking.

- **Real-Time Processing**

Providing robust mechanisms for real-time updates, including order status changes, menu modifications, and rider location tracking.

These challenges will be addressed using a modular and iterative development approach (Agile-Iterative Model), which allows incremental feature addition and continuous improvement based on user feedback. This approach ensures flexibility, early validation, and alignment with user expectations while mitigating risks associated with large-scale deployments.

2.1 Problem Statement

The rapid growth of online food delivery platforms has transformed the way people order and receive meals, yet these systems largely cater to established restaurants and commercial food providers. As a result, home-based cooks, small-scale food vendors, and independent delivery riders remain underserved and often excluded from mainstream digital marketplaces. These individuals lack a structured platform through which they can showcase their meals, manage orders, handle payments, or build a sustainable income stream. Customers who prefer fresh, homemade food similarly face difficulties finding reliable home cooks, verifying food quality, placing secure orders, and tracking delivery progress in real time. This gap creates a fragmented market where demand exists but is not effectively connected with supply.

Existing platforms also face significant limitations in real-time communication, transparency, and workflow management. Customers often experience delayed updates, cooks struggle to manage menu changes or order statuses efficiently, and riders receive incomplete information regarding delivery tasks. Additionally, most available systems do not support three-way coordination among customers, cooks, and riders, resulting in miscommunication, delivery delays, and poor user satisfaction. The absence of dedicated tools for earnings management, delivery tracking, review systems, and dispute resolution further affects the reliability and convenience of current solutions.

Backend issues also contribute to the problem. Many platforms lack the scalability required to handle growing user bases, the modularity needed to support multiple roles, and the real-time processing capabilities essential for dynamic food delivery operations. These technical shortcomings reduce system performance and undermine the user experience. Without a robust architecture, existing systems struggle to maintain accuracy in order states, ensure secure transactions, or provide consistent updates across all user roles.

The foodO platform has been conceived as a solution to these challenges by creating a unified digital ecosystem that connects home cooks, customers, and riders through a seamless workflow. The goal is to provide reliable access to home-cooked meals, empower home cooks with business management tools, and offer riders accessible earning opportunities. foodO aims to address existing

gaps by introducing a transparent, scalable, and real-time platform capable of supporting end-to-end food delivery operations. Through structured workflows, automated processes, and clear communication channels, the system seeks to resolve long-standing issues of inconsistency, inefficiency, and lack of digital support in the home-cooked food marketplace.

2.2 Deliverables and Development Requirements

The development of the foodO platform is organized around a well-defined set of deliverables and technical requirements that collectively ensure the creation of a robust, scalable, and user-friendly system. These deliverables are aligned with the project's objective to provide a comprehensive solution for home-cooked food delivery, integrating multiple user roles—Customer, Cook, Rider, and Admin—within a unified web-based environment. Each deliverable targets a specific functional area, while the development requirements outline the technologies, tools, and standards necessary for successful implementation.

2.2.1 Key Deliverables

Backend System (Node.js, Express.js, MongoDB)

- RESTful API architecture with secure route handling for all user roles.
- JWT-based authentication for Customers, Cooks, Riders, and Admins.
- CRUD operations for menu items, user profiles, orders, and reviews.
- Real-time order status updates and notifications using Socket.IO.
- Image upload and management for food items integrated with cloud/local storage.
- Modular structure for controllers, routes, and models to ensure scalability and maintainability.
- Integration of Agile-Iterative development for progressive feature rollouts.

Customer Web Application (React.js)

- Responsive web interface for browsing menus and placing orders.
- User registration, login, and profile management.
- Menu browsing with filtering and search options.
- Cart functionality with item quantity management and secure checkout.
- Real-time order tracking and status updates.
- Review and rating system for dishes and cooks.

Cook Dashboard (React.js)

- Secure login and profile management for home chefs.

- Menu management: Add, update, and delete food items with images and descriptions.
- Order monitoring: View pending, accepted, and completed orders.
- Stock control and availability toggle for menu items.
- Earnings overview and withdrawal request functionality.

Rider Dashboard (React.js)

- Rider authentication and profile management.
- Delivery assignment acceptance and status updates (Picked Up, Delivered).
- Real-time location sharing for order tracking.
- Earnings summary and withdrawal request feature.

Admin Panel

- Manage users (Customers, Cooks, Riders) and monitor platform activities.
- Approve or reject withdrawal requests.
- System analytics and reporting for performance monitoring.

2.2.2 Development Requirements

To implement the above deliverables, the following technical, and environmental requirements must be met:

Technical Requirements

- Node.js and Express.js as the primary backend framework for building RESTful APIs.
- MongoDB for document-oriented database storage to handle dynamic data like menus and orders.
- React.js for developing the responsive web-based interface for all user roles.
- Socket.IO for real-time communication and order status updates.
- Stripe API for secure payment processing.
- Google Maps API for geolocation and delivery tracking.
- Postman for API testing and validation.
- Git and GitHub for version control and collaborative development.
- UUID for generating unique identifiers for images and records.
- JWT (JSON Web Tokens) for secure authentication and session management.

Environmental Requirements

- **Development Environment:** Visual Studio Code (VS Code) as the primary IDE.
- **Testing Environment:** Local development server with Node.js and React, and Postman for API testing.
- **Database Hosting:** MongoDB Atlas for cloud-based database or locally hosted MongoDB instance with replica sets for reliability.
- **Deployment Environment:** Cloud hosting platforms such as AWS, Vercel, or Heroku for scalability and high availability.
- **Security Setup:** HTTPS configuration and SSL certificates for secure communication.

2.3 Current System

The current food delivery ecosystem relies heavily on commercial applications designed primarily for large restaurants and established food businesses. While these platforms provide basic ordering and delivery workflows, they do not adequately support home-based cooks, independent riders, or customers seeking fresh, homemade meals. This gap has resulted in a fragmented, inefficient environment where communication is inconsistent, order management is often unclear, and essential features for small vendors are missing entirely. The absence of a unified and transparent system creates challenges across all user roles, making it difficult for cooks to scale their offerings, riders to streamline their work, and customers to enjoy a smooth, reliable ordering experience. The following subsections highlight the shortcomings of the existing ecosystem that foodO aims to address.

2.3.1 Lack of Integration in Existing Food Delivery Systems

In most existing platforms, the workflow between customers, food providers, and delivery personnel is not fully integrated. Traditional food delivery systems are optimized for restaurants with automated kitchen operations and pre-structured menus, leaving no room for the unique workflows of home-based cooks. These cooks operate without access to tools for structured menu management, earnings monitoring, inventory handling, or real-time order visibility. Riders on these platforms also receive limited support, as they often rely on separate communication channels to coordinate deliveries, which creates delays, confusion, and inaccuracies in delivery timelines. Additionally, customers who prefer home-cooked meals struggle to find trustworthy home chefs due to the absence of a dedicated, integrated marketplace. This lack of cohesion leads to inefficiencies, miscommunication, and operational bottlenecks that disrupt the overall experience for all stakeholders.

2.3.2 User Experience and Quality Challenges

Current food delivery solutions often present several user experience challenges that directly impact both customers and home-based vendors. Customers commonly face issues such as incomplete meal listings, inconsistent order updates, and limited transparency regarding preparation and delivery progress. Without clear communication, customers are left uncertain about when their meal will be prepared or delivered. Home cooks, on the other hand, lack structured tools to manage their digital presence, resulting in disorganized menus, irregular item updates, and slow responses to customer orders. Riders also encounter difficulties due to insufficient route guidance and unclear delivery instructions. These fragmented workflows reduce overall platform reliability and diminish user confidence, ultimately affecting satisfaction and repeat usage.

2.3.3 Manual Menu and Order Management

In the absence of a specialized platform, home cooks and small-scale food vendors rely heavily on manual processes to manage their operations. Menu updates are often communicated through social media platforms, messaging apps, or informal channels, making it difficult to maintain accuracy and consistency. Order placement typically happens through chat-based communication, which leads to human errors, delayed responses, and challenges in tracking ongoing orders. Riders also depend on direct messaging to receive delivery details, which results in frequent miscommunication, especially when multiple orders need to be managed simultaneously. These manually managed processes are not scalable and become increasingly inefficient as order volume grows. The lack of automation creates several operational risks, including lost orders, incorrect deliveries, incomplete transaction records, and limited reporting capabilities for cooks and riders.

foodO addresses these limitations by offering an automated, structured, and integrated system that eliminates the reliance on manual processes and ensures smoother coordination among all participants in the home-cooked food delivery pipeline.

Chapter 3

Requirement Analysis

3 Requirement Analysis

Derived from the SRS, this chapter describes how the system's needs were gathered, documented, and represented.

3.1 Use Cases Diagram(s)

The foodO system includes use case diagrams for its three primary interacting roles: Customer, Cook, and Rider. Each actor performs a set of actions necessary for ordering, preparing, and delivering home-cooked meals. These diagrams represent how different users interact with the system to achieve their goals and how the platform supports end-to-end order processing, beginning from menu browsing to order delivery completion.

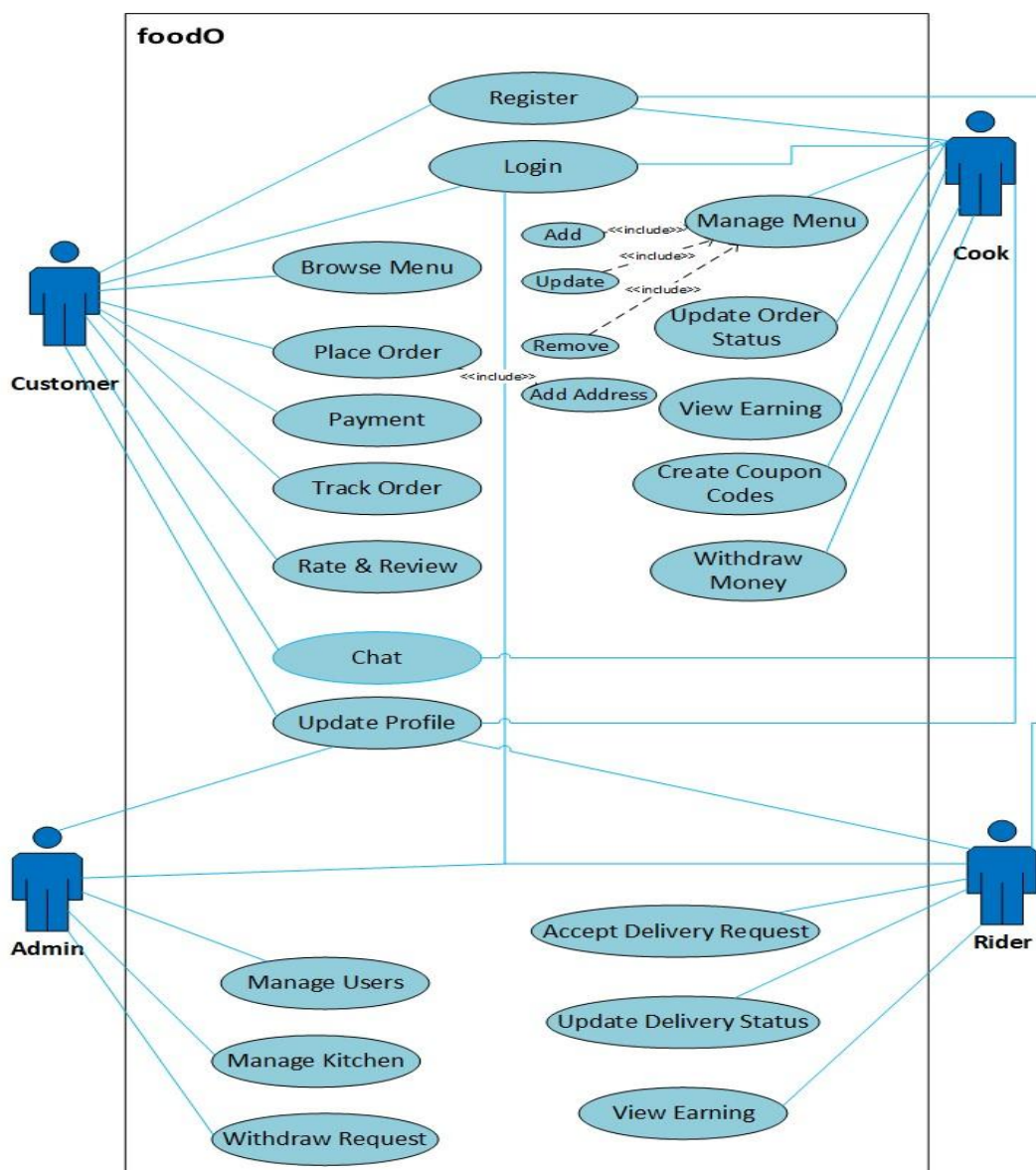


Figure 3. 1 Use Case

1. Overview of the Use Case Diagram

The use case model for foodO illustrates how different actors engage with the system to perform their required operations. It visualizes system behavior from a user's perspective, showing the interactions between Customers, Cooks, and Riders. Each oval in the diagram represents a system function, while the external actors indicate the users who initiate those functions. The system boundary labeled "foodO" encloses all internal functionalities provided by the platform.

2. Primary Actors in the System

Customer

The Customer is the primary end-user who browses available meals, places orders, tracks preparation and delivery, rates food, and manages their personal profile.

Cook

The Cook is a home-based vendor responsible for creating and managing menu items, accepting orders, preparing meals, updating preparation status, and managing earnings.

Rider

The Rider is responsible for receiving delivery requests, picking up meals from cooks, delivering orders to customers, and updating delivery statuses throughout the process.

3. Use Cases for the Customer

The **Customer** actor is connected with the following use cases:

a) Register

Customers create an account by entering basic information such as name, email, phone number, and password.

b) Login

Once registered, customers authenticate using login credentials to access their dashboard.

c) Browse Menu

Customers explore available food items listed by various home cooks, including viewing prices, descriptions, and categories.

d) Place Order

Customers select items, add them to the cart, and proceed to checkout. The process includes reviewing order details and confirming the purchase.

e) Enter Delivery Details

Customers provide delivery address, contact number, and optional notes for the rider.

f) Track Order

The system enables real-time tracking of the order's status during preparation and delivery.

g) Rate and Review Meal

After receiving an order, customers can submit ratings and written reviews for the cook and delivery experience.

h) Update Profile

Customers may update their personal information, including contact details and saved addresses.

4. Use Cases for the Cook

The **Cook** in the foodO system manages menu items, views incoming customer orders, and updates order statuses throughout the preparation process. The cook can also monitor earnings, handle withdrawal requests, and review customer feedback to maintain service quality. These features enable home-based cooks to efficiently run their food operations through a structured and reliable digital workflow.

a) Manage Menu

Cooks create, edit, and delete meal items. They can update prices, descriptions, and availability according to their daily offerings.

b) View & Accept Orders

Cooks receive order notifications, view order details, and accept or decline incoming requests.

c) Update Order Status

Cooks update statuses such as *Accepted*, *In Preparation*, and *Ready for Pickup*, enabling real-time tracking for customers and riders.

d) View Reviews

Cooks can access feedback left by customers to gauge satisfaction and improve quality.

e) View Earnings

Cooks view their earnings history, pending payouts, and financial summaries.

f) Withdraw Money

Cooks can request a withdrawal of their available balance using integrated payment gateways such as PayPal.

5. Use Cases for the Rider

a)Accept Delivery Request

Riders receive delivery requests once the cook marks an order as ready. They can choose to accept or decline.

b)Pickup Order

Upon accepting, riders pick up the prepared meal from the cook's location.

c)Update Delivery Status

Riders update statuses like *Picked Up*, *On the Way*, and *Delivered*, ensuring real-time visibility for customers.

d)View Delivery History

Riders may view completed deliveries and total earnings.

e)Withdraw Earnings

Riders can withdraw funds to their accounts through supported payment platforms.

6. Shared Use Cases

Certain use cases involve interactions among multiple actors:

- **Place Order** requires the cook to accept and prepare the order.
- **Track Order** depends on both cook updates (preparation) and rider updates (delivery).
- **Update Order Status** involves both cooks and riders during different workflow stages.
- **Rate & Review** connects customer feedback with cook performance evaluation.

7. Relationships and Connectors

- **Solid lines** connect actors to the use cases they initiate.
- **<<include>>** relationships indicate mandatory sub-processes (e.g., *Enter Delivery Details* included in *Place Order*).
- **System boundary** clearly separates foodO functionalities from external users.

8. Functional Coverage of the System

This use case diagram comprehensively covers the functionality of a food ordering system:

Table 3. 1 Feature Category for use case

Feature Category	Description
User Authentication	Registration and login for customers, cooks, and riders
Menu Navigation	Customers browse and search for meals
Order Lifecycle Management	Ordering, preparation updates, delivery tracking
Cook Operations	Menu management, earnings, order updates
Rider Operations	Accepting deliveries, updating statuses, withdrawals
Review System	Customer feedback and cook evaluation

9. Benefits of This Use Case Diagram

The use case diagram helps identify user responsibilities, supports planning and modular development, and serves as a foundation for functional requirements and test case generation. It also clarifies interactions among customers, cooks, and riders, ensuring that the platform supports a complete and coherent food delivery ecosystem.

10. Potential Enhancements

Possible refinements include adding the **Admin** actor, integrating **in-platform chat**, and detailing features such as **payment processing**, **notifications**, and **customer support flows** to increase clarity and system completeness.

3.2 Detailed Use Case

The tables below indicate a comprehensive use case template filled out for the all use cases in the foodO system.

Table 3. 2 Login use case

Use Case ID	01
Use case name	Login
Actors	- Customer

	- Cook - Rider
Description	A user logs into their foodO account using credentials. The system authenticates login info and provides access to the user dashboard.
Trigger	User clicks the "Login" button on the welcome screen.
Pre-condition	The user has already registered and holds valid login credentials.
Postcondition	The user is redirected to their dashboard with access to their functionalities.
Normal flow	- User enters email/phone and password. - System checks credentials. - If valid, user is logged in.
Alternative flow	- User enters incorrect details. - System shows an error and asks to retry.
Exceptions	- Incorrect password/email. - No internet connection. - Backend server timeout.
Business rules	- Login requires internet access. - Email/phone must match stored record. - Multiple failed attempts may lock the account.

Table 3. 3 Register use case

Use Case ID	02
Use case name	Register
Actors	- Customer - Cook - Rider
Description	New users provide personal information to create a foodO account.
Trigger	User clicks the "Register" button on the home screen.
Pre-condition	User is not already registered with the same credentials.
Postcondition	User account is successfully created in the system.

Normal flow	- User fills registration form. - System validates inputs. - Account is created.
Alternative flow	- Email/phone already used. - Error message prompts correction.
Exceptions	- Network issue - Required fields left blank.
Business rules	- Email/phone must be unique. - All mandatory fields must be filled.

Table 3. 4 Browse Menu use case

Use Case ID	03
Use case name	Browse Menu
Actors	Customer
Description	Customers explore food items listed by cooks with filtering and search options.
Trigger	Customer clicks on "Browse Menu" from the home page.
Pre-condition	Menu items must be added by cooks and marked available.
Postcondition	Menu list is displayed to the customer.
Normal flow	- System fetches active menu. - Menu is displayed with filter/sort tools. - Customer views details.
Alternative flow	No items available – system shows a message.
Exceptions	Menu fetch failure due to network error.
Business rules	- Only available items are shown. - Prices must be visible.

Table 3. 5 Place Order use case

Use Case ID	04
Use case name	Place Order
Actors	Customer

Description	Customer selects menu items, adds address, and submits the order. The system confirms the order and notifies the cook.
Trigger	Customer clicks “Place Order” after adding items to cart.
Pre-condition	Customer is logged in and has items in the cart.
Postcondition	Order is saved and sent to cook for preparation.
Normal flow	- Select items - Add address - Submit order - Get confirmation.
Alternative flow	- Cart is empty – show error - Address missing – prompt to add.
Exceptions	- System crash - Internet issue during order confirmation.
Business rules	- Minimum one item required - Valid address is mandatory.

Table 3. 6 Payment use case

Use Case ID	05
Use case name	Payment
Actors	Customer
Description	User completes payment using the preferred method. System validates and processes the transaction.
Trigger	Customer chooses “Pay Now” after placing the order.
Pre-condition	Order is placed and payment method is available.
Postcondition	Payment is processed, and status is updated.
Normal flow	- Choose method - Enter payment info - Process and confirm.
Alternative flow	Payment declined – retry or choose different method.
Exceptions	- Gateway timeout - Card/network error.
Business rules	- Must use a valid and accepted method - Secure payment required.

Table 3. 7 Track Order use case

Use Case ID	06
Use case name	Track Order
Actors	Customer
Description	Customer shares feedback on the food and cook's service after delivery.
Trigger	After placing the order, customer accesses "Track Order."
Pre-condition	Order must be placed and in progress.
Postcondition	The system displays products that match the selected filter criteria.
Normal flow	- Open order tracking - View real-time status updates - Estimated time shown.
Alternative flow	No location data – fallback to status message.
Exceptions	- GPS error - No internet.
Business rules	- Rider's device must share location - Secure tracking required.

Table 3. 8 Rate & Review use case

Use Case ID	07
Use case name	Rate & Review
Actors	Customer
Description	Customers can track the status and location of their food order from the moment it is placed until it is delivered.
Trigger	Order is delivered and customer clicks "Rate Order."
Pre-condition	Order must be marked as delivered.
Postcondition	Rating is saved and visible to cook.

Normal flow	- Customer selects stars - Enters review - Submits feedback.
Alternative flow	Skips rating – no review saved.
Exceptions	- Feedback form error - Network failure.
Business rules	- One review per order - Profanity filter applied.

Table 3. 9 Chat use case

Use Case ID	08
Use case name	Chat
Actors	- Customer - Cook - Rider
Description	Allows real-time messaging between users related to orders and delivery.
Trigger	User selects “Chat” from order details or menu.
Pre-condition	Both users must be logged in.
Postcondition	Message is sent and received.
Normal flow	- Open chat window - Type and send message - Receive reply.
Alternative flow	User not online – show pending status.
Exceptions	Messages not sent due to connection issue.

Business rules	- Chat is encrypted - Order-related only chats allowed.
----------------	--

Table 3. 10 Update Profile use case

Use Case ID	09
Use case name	Update Profile
Actors	- Customer - Cook - Rider
Description	Users can update their profile details and preferences anytime.
Trigger	Click on “Profile” and choose “Edit.”
Pre-condition	User is logged in.
Postcondition	Profile changes are saved.
Normal flow	- Open profile - Modify fields - Save changes.
Alternative flow	Invalid inputs – show validation errors.
Exceptions	Database update failure.
Business rules	- Unique phone/email required - Secure access only.

Table 3. 11 Manage Menu use case

Use Case ID	10
Use case name	Manage Menu
Actors	Cook
Description	Cook manages items on their food menu.
Trigger	Cook selects 'Manage Menu'.
Pre-condition	Cook is logged in.
Postcondition	Menu items are added, updated, or removed.
Normal flow	- Cook adds/updates/removes menu items. - System saves changes.

Alternative flow	- Duplicate item name. - Prompt change.
Exceptions	- Invalid input data. - Database failure.
Business rules	Items must include name, price, description.

Table 3. 12 Update Order Status use case

Use Case ID	11
Use case name	Update Order Status
Actors	Cook
Description	Cook updates the status of a customer's order.
Trigger	Cook selects an order and changes its status.
Pre-condition	Order is assigned to the cook.
Postcondition	Status reflects current order state.
Normal flow	- Cook updates status: Preparing, Ready. - System logs update.
Alternative flow	- Wrong status selected. - Cook corrects update.
Exceptions	System error while saving status.
Business rules	Status change is timestamped.

Table 3. 13 View Earning use case

Use Case ID	12
Use case name	View Earning
Actors	- Cook - Rider
Description	Cook or rider views their earnings.
Trigger	User clicks 'View Earning'.
Pre-condition	User is logged in.

Postcondition	Earning details are shown.
Normal flow	- System fetches earnings from database. - Displays per order/day/month.
Alternative flow	- No earnings yet. - Display message.
Exceptions	Database error.
Business rules	Only completed orders are counted.

Table 3. 14 Create Coupon Codes use case

Use Case ID	13
Use case name	Create Coupon Codes
Actors	Cook
Description	Cook creates promotional discount coupons.
Trigger	Cook clicks 'Create Coupon'.
Pre-condition	Cook has access rights.
Postcondition	Coupon is saved and usable.
Normal flow	- Cook enters code, discount. - System saves code.
Alternative flow	- Duplicate code. - Prompt error.
Exceptions	Invalid discount value.
Business rules	Coupons must be unique and valid within a date range.

Table 3. 15 Withdraw Money use case

Use Case ID	14
Use case name	Withdraw Money
Actors	Cook
Description	Cook withdraws accumulated earnings.

Trigger	User selects 'Withdraw Money'.
Pre-condition	Earnings are available for withdrawal.
Postcondition	Request is submitted to admin.
Normal flow	- Cook enters amount. - System sends request to admin.
Alternative flow	- Insufficient funds. - Error displayed.
Exceptions	Transaction error.
Business rules	Minimum balance must remain.

Table 3. 16 Accept Delivery Request use case

Use Case ID	15
Use case name	Accept Delivery Request
Actors	Rider
Description	Rider accepts order delivery assignments.
Trigger	Rider clicks 'Accept'.
Pre-condition	There is a pending delivery request.
Postcondition	Rider is assigned to delivery.
Normal flow	- Rider views pending orders. - Accepts request.
Alternative flow	- Already accepted by another. - System notifies rider.
Exceptions	Server update delay.
Business rules	Only one rider per order.

Table 3. 17 Update Delivery Status use case

Use Case ID	16
Use case name	Update Delivery Status
Actors	Rider

Description	Rider updates order delivery progress.
Trigger	Rider changes order status.
Pre-condition	Rider is assigned an order.
Postcondition	Updated status is reflected for customer.
Normal flow	- Rider marks: Picked Up, On The Way, Delivered. - System updates status.
Alternative flow	- Missed status change. - Rider corrects later.
Exceptions	Network failure.
Business rules	Status must be sequential.

Table 3.18 Manage Users use case

Use Case ID	17
Use case name	Manage Users
Actors	Admin
Description	Admin manages customer, cook, and rider accounts.
Trigger	Admin selects 'Manage Users'.
Pre-condition	Admin is logged in.
Postcondition	User data is updated or deleted.
Normal flow	- Admin views list. - Edits or deletes user.
Alternative flow	- User not found. - Prompt displayed.
Exceptions	Permission error.
Business rules	Only admin can manage users.

Table 3.19 Manage Kitchen use case

Use Case ID	18
Use case name	Manage Kitchen

Actors	Admin
Description	Admin manages kitchen profiles and data.
Trigger	Admin selects 'Manage Kitchen'.
Pre-condition	Admin is logged in.
Postcondition	Kitchen status and info updated.
Normal flow	- Admin reviews kitchen data. - Edits or disables kitchen.
Alternative flow	- Kitchen not found. - Prompt error.
Exceptions	Database error.
Business rules	Kitchen operations linked to cooks.

Table 3.20 Withdraw Request use case

Use Case ID	19
Use case name	Withdraw Request
Actors	Admin
Description	Admin reviews and processes withdrawal requests submitted by cooks.
Trigger	Admin clicks on 'Withdraw Request' section.
Pre-condition	There are pending withdrawal requests.
Postcondition	Request is marked as approved or rejected.
Normal flow	- Admin views all pending requests. - Admin approves or rejects each. - System updates request status.
Alternative flow	- Request already processed. - Notify admin.
Exceptions	- Database error. - Request data corrupted.
Business rules	Only verified requests are processed.

3.3 Functional Requirements

The functional requirements of the foodO system describe the core actions that each user role must be able to perform within the platform. Since foodO supports four primary actors—Customer, Cook, Rider, and Admin—the system ensures that each module contains clearly defined functionalities to support efficient interaction, smooth order flow, and reliable food delivery operations.

3.3.1 Customer Module

The Customer module allows users to interact with the platform by performing essential actions related to food browsing, ordering, and communication. A customer must be able to register and log in to the system, browse menus from available cooks, search for specific food items, and view detailed dish descriptions. The customer should be able to add items to the cart, place an order, enter delivery information, and make online payments securely. After placing an order, the customer can track its progress in real time, from acceptance to delivery. The system also enables customers to provide reviews and ratings for both meals and delivery experiences, update their profile information, and access their complete order history.

3.3.2 Cook Module

The Cook module provides home-based food providers with tools to manage their food offerings and process customer orders. A cook must be able to register and create a profile, which may be approved by the admin. Once active, the cook can add new meals, update existing menu items, and manage availability based on ingredients or daily schedules. The cook can view all incoming orders, accept or reject them, and update the order status throughout preparation. The system also provides cooks with access to customer reviews, earnings summaries, withdrawal requests, and order history, enabling them to operate efficiently within the platform.

3.3.3 Rider Module

The Rider module focuses on enabling delivery personnel to receive, manage, and complete delivery tasks assigned through the system. Riders must be able to register, log in, and maintain their profile information. They can view available delivery requests, accept or decline them based on availability, and follow the delivery route using integrated navigation tools. Riders can update delivery statuses such as “Picked Up,” “On the Way,” and “Delivered,” ensuring real-time tracking for customers and cooks. They also have access to their daily earnings, completed deliveries, and withdrawal management for payouts.

3.3.4 Admin Module

The Admin module provides centralized control over the entire foodO platform. The admin manages user registrations, verifies cook and rider accounts, handles reported issues, and monitors overall system activity. The admin can manage categories, review all customer orders, and ensure smooth system operations by monitoring order flows and resolving disputes when necessary. The admin also oversees financial processes such as withdrawal approvals, manages platform-wide settings, and ensures compliance with operational guidelines. Through this module, the admin maintains stability, quality, and security across the entire foodO ecosystem.

3.4 Non-Functional Requirements

The system also incorporates several key non-functional requirements:

Usability

The platform provides a clean, intuitive, and responsive web interface designed to support smooth navigation for all users, including individuals with minimal technical experience. Consistent layout, clear visual hierarchy, and accessible design principles ensure that customers, cooks, and riders can easily perform tasks such as browsing menus, managing orders, and updating delivery statuses.

Performance

foodO is optimized to deliver fast response times and seamless real-time updates. Efficient backend processes and structured data handling minimize delays during menu fetching, order placement, and delivery tracking. The system is designed to handle multiple simultaneous user requests without compromising speed or stability.

Portability

As a web-based platform built using modern frontend technologies, foodO operates smoothly on major browsers and devices, including desktops, laptops, tablets, and mobile browsers. This browser-based accessibility eliminates the need for separate mobile applications while ensuring consistent performance and user experience across all screen sizes.

Scalability

The system architecture is designed to accommodate increasing numbers of customers, cooks, riders, menu items, and orders. Modular backend components and a flexible database structure allow the platform to expand without affecting overall system performance. foodO can easily integrate new features, roles, or services as the platform grows.

Security

The system ensures secure user authentication, data protection, and safe transaction processing. Sensitive information such as passwords and payment details is protected through encryption and secure communication protocols, ensuring a safe environment for all users.

Reliability

foodO maintains stable, error-free operation through robust server-side processing and real-time data synchronization mechanisms. The system is designed to minimize downtime and maintain order accuracy, even during high traffic periods.

Chapter 4

Design and Architecture

4 Design and Architecture

From the SDD, this chapter covers the structural and technical design of the system.

4.1 System Architecture

The architecture of foodO follows a standard client–server model, where all user roles (Customer, Cook, Rider, Admin) interact with the system through a web-based client, while the backend processes requests, applies business rules, and communicates with the database and external services. The architecture is divided into four main layers: Client Side, Server Side, Database Layer, and External Services. This layered structure ensures modularity, maintainability, and scalable communication between all components.

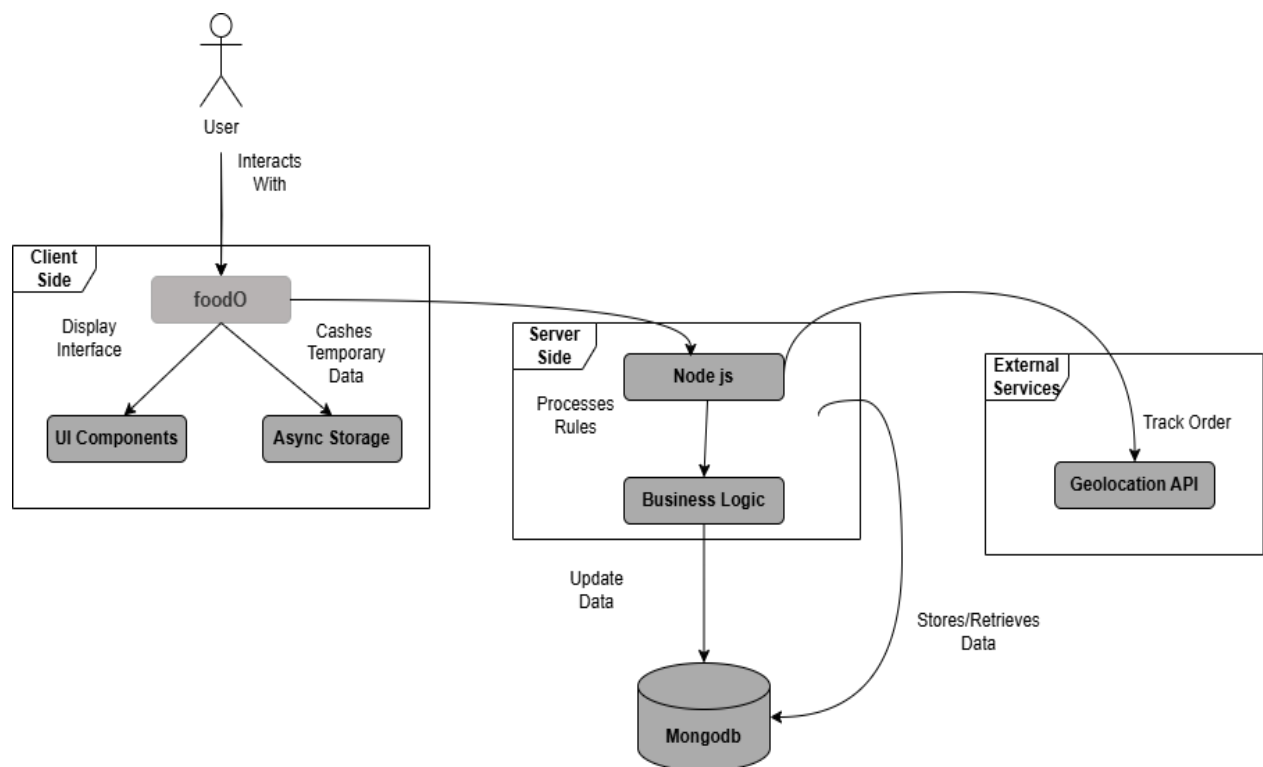


Figure 4. 1 Architecture Diagram

1. Introduction to System Architecture

The architecture of foodO is designed to handle multiple user roles efficiently while maintaining smooth communication between the frontend and backend. All requests—such as logging in, placing orders, viewing menus, managing deliveries, or accessing earnings—flow through clearly separated layers. Each layer is responsible for a specific part of the system’s behavior, ensuring clarity and ease of future expansion.

2. Overview of Components

The architecture consists of:

- **Client Side:** The web interface displaying UI components and temporarily storing session data.
- **Server Side:** The backend powering business logic, validations, and communication.
- **Database Layer:** MongoDB storing users, cooks, riders, menus, orders, reviews, and payouts.
- **External Services:** Third-party APIs for geolocation, delivery tracking, and optional notifications.

Each module performs distinct tasks while interacting seamlessly as a unified ecosystem.

3. Client Side

The Client Side represents the foodO web application that users interact with. Whether the user is a Customer browsing meals, a Cook updating menus, a Rider viewing delivery tasks, or an Admin managing users, the web interface ensures consistent interaction across devices.

a) User Interaction

All system workflows begin with user actions through the foodO interface. These include browsing food items, placing orders, approving orders, accepting deliveries, or generating reports.

b) UI Components

The UI is composed of structured components such as:

- Buttons, forms, search bars
- Order cards and tracking interfaces
- Menu creation and earnings dashboards
- Admin management panels

These components provide responsive, real-time interaction with the backend.

c) Local/Async Storage

The client may temporarily store:

- Authentication tokens
- Cart items
- Session-related preferences

This improves response time and prevents data loss in case of page refresh or temporary network drop.

d) Responsibilities

The client is responsible for:

- Displaying data received from the backend
- Capturing user actions
- Sending requests via REST APIs
- Maintaining lightweight cached data

4. Server Side

The **Server Side** is the core of the platform, responsible for processing logic, handling requests, managing authentication, updating order statuses, and communicating with the database.

a) Node.js Backend

The backend is developed using Node.js due to its non-blocking I/O model, making it ideal for applications requiring real-time updates such as delivery tracking and order confirmations.

It receives requests for:

- Registration/Login
- Menu updates
- Order placement and status changes
- Delivery acceptance
- Review submission
- Admin management tasks

b) Business Logic Layer

This layer executes all system rules, including:

- Verifying credentials and permissions
- Generating and validating JWT tokens
- Handling cart processing and order creation

- Assigning riders and tracking delivery progress
- Updating cook earnings and payout requests
- Validating menu inputs and product availability

c) Communication With Database

Once the business logic validates and processes the request, it fetches or updates data in MongoDB.

d) Responsibilities of Backend

- Routing API requests
- Applying validation and security rules
- Ensuring real-time communication with riders and customers
- Managing notifications and external API calls

5. Database Layer – MongoDB

MongoDB stores all the dynamic data used in the system. Its flexible schema and document-based storage make it ideal for a platform with varying food items, delivery statuses, and user profiles.

Stored Collections Include:

- Customers
- Cooks
- Riders
- Admin users
- Menu Items
- Orders
- Reviews
- Transactions & Withdrawals

a) Flow of Data

1. Client sends request → Server validates and processes it.
2. Backend reads/writes data in MongoDB.

3. Server returns structured responses to the frontend.

This process ensures a fast and consistent data flow.

6. External Services

The architecture also supports integration with external APIs needed to enhance platform functionality.

a) Geolocation API

- Used to:
- Track rider routes
- Estimate delivery time
- Provide location updates to customers

b) Optional Future Integrations

- Email/SMS notifications
- Payment services such as PayPal or Stripe
- Mapping and route optimization tools

7. How Components Work Together: Example Flow

To understand how the components work together, consider the user journey:

1. Customer browses food on the web interface.
2. Client sends an API request to the backend to create an order.
3. Backend validates the order and saves it in MongoDB.
4. Cook receives the order and updates its status.
5. Rider accepts delivery through the rider panel.
6. Geolocation provider updates delivery location.
7. Customer sees real-time status until the food is delivered.
8. Order is completed and stored for future reporting.

This demonstrates the smooth communication among all layers.

8. Key Benefits of This Architecture

Table 4. 1 Benefits of Architecture Diagram

Feature	Benefit
Modular Components	Easy maintenance and scalable upgrades.
Node.js Backend	Fast processing and real-time handling of multiple users.
MongoDB Storage	Flexible schema suited for dynamic orders and menus.
External APIs	Efficient tracking, payments, and notifications.
Client-side Caching	Faster performance and smoother user experience.

9. Future Enhancements

Potential improvements include:

- Integrated notification system (SMS, email, push alerts).
- AI-based food recommendations.
- Automated rider route optimization.
- Predictive analytics dashboards for cooks and admin.

4.2 Data Representation [Diagram + Description]

The database contains multiple collections/tables

- **Users**

Stores user login details and basic personal information.

- **Customers**

Holds customer identification and contact details such as custID and phone.

- **Address**

Stores delivery location information including phone and city.

- **Menu Items**

Contains food item details such as name, description, price, and reviews.

- **Orders**

Stores customer order information including orderID, kitchen, and item associations.

- **Cooks**

Maintains information about which cook prepares each order.

- **Delivery**

Handles delivery assignment details, connecting orders with address and rider information.

- **Reviews**

Stores customer feedback related to menu items.

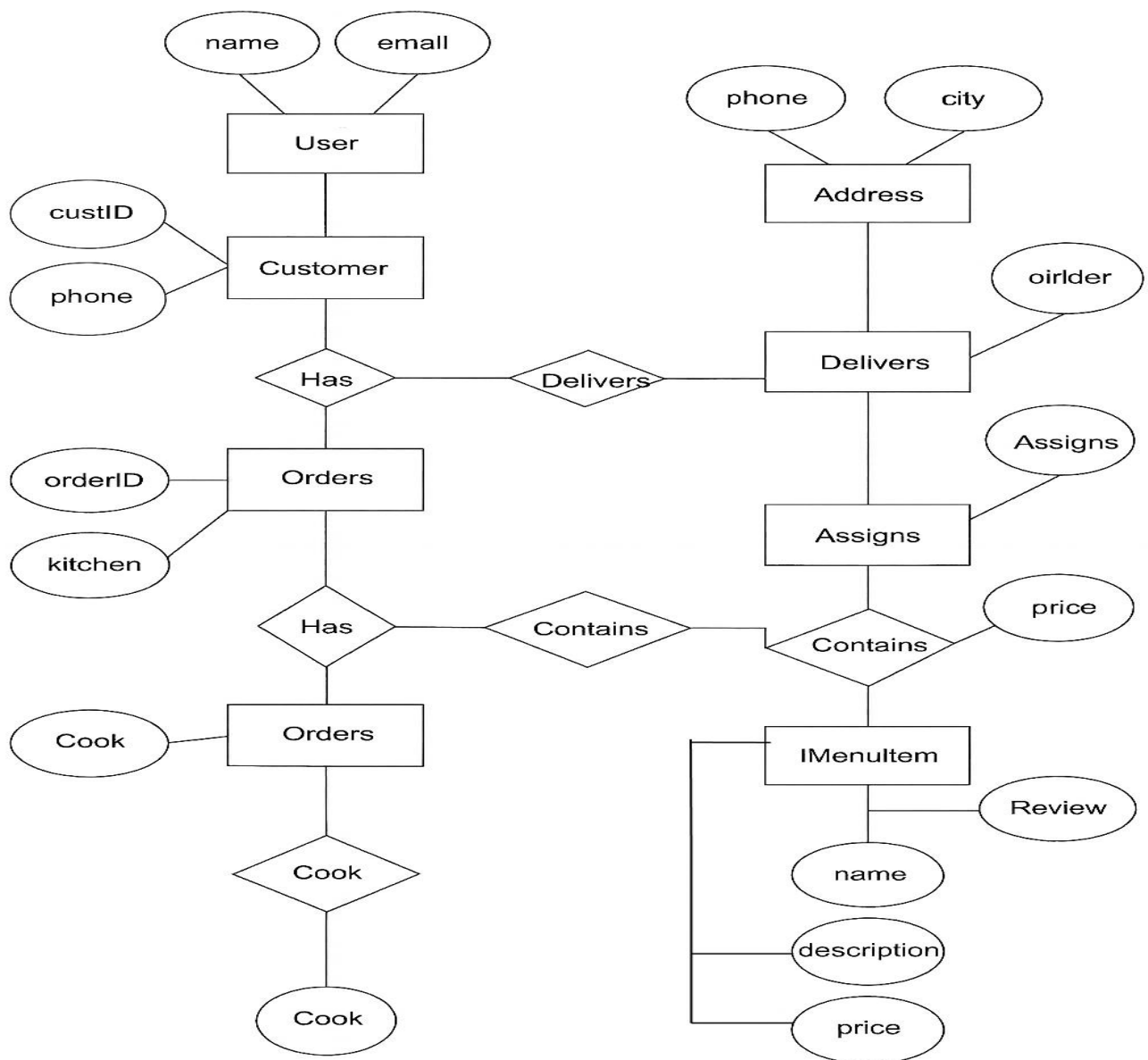


Figure 4. 2 ER Diagram

1. Introduction

This ERD illustrates the logical data model of an online food ordering and delivery application. The diagram contains the main entities involved in food ordering, delivery assignment, menu management, and user identification.

2. Entity: User

The *User* entity stores basic login and identification information for individuals using the system.

Attributes:

- **name**
- **email**

A user is directly connected to a **Customer**, indicating that a registered user becomes a customer in the system.

3. Entity: Customer

The **Customer** entity represents the person placing food orders.

Attributes:

- **custID**
- **phone**

Relationships:

- Connected to **User**
- Linked to **Orders** through the relationship **Has**

A customer can place multiple orders.

4. Entity: Address

The Address entity stores the customer's delivery location.

Attributes:

- **phone**

- **city**

Relationships:

- Connected to **Delivers**, indicating an order is delivered at this address.

5. Entity: Orders

The **Orders** entity represents the food orders made by customers.

Attributes:

- **orderID**
- **kitchen**

Relationships:

- A Customer **Has** Orders
- Orders **Contains** Menu Items (through the Contains relationship)
- Orders are linked to the **Cook** through a Cook relationship
- Orders participate in the **Delivers** relationship

Each order may contain multiple menu items and is handled by a cook

6. Entity: MenuItem

Stores information about a specific food item.

Attributes:

- **name**
- **description**
- **price**
- **Review**

Relationships:

- Linked to Orders through **Contains**
- Connected to Assigns
- Attributes description and price are correctly connected in your marked diagram

7. Relationship: Delivers (Orders → Address)

This shows that:

- Every **Order** is delivered to one **Address**
- Each delivery is associated with a **Delivers** entity (delivery process)

8. Relationship: Delivery

This indicates:

- An **Order** may contain many **MenuItems**
- A **MenuItem** may be part of many **Orders**

It represents a many-to-many type linkage.

9. Relationships Summary

Table 4. 2 Relationships Summary

Relationship	Entities Involved	Type	Explanation
User → Customer	User, Customer	1–1	A User account corresponds to one Customer profile.
Customer → Orders	Customer, Orders	1–Many	A customer can place multiple orders.
Orders → MenuItem	Orders, MenuItem	Many–Many	Using Contains relationship.
MenuItem → Review	MenuItem, Review	1–Many	Each menu item can have multiple reviews.
Orders → Cook	Orders, Cook	Many–One	An order is prepared by one cook.
Orders → Address (Delivers)	Orders, Address	Many–One	Each order is delivered to one address.
Delivers MenuItem (Assigns) →	Delivers, MenuItem	Many–Many	Assignment relationship in delivery.

10. Normalization

The FoodO database design adheres to the core principles of **Third Normal Form (3NF)**, ensuring a clean, efficient, and logically structured data model.

- **No repeated attributes:**

Each entity contains unique, non-redundant attributes. For example, custID, phone, city, name, description, and price appear only once in their respective entities.

- **Atomic values:**

All attributes store atomic (indivisible) values such as custID, city, price, orderID, and name. No multi-valued or composite repeating fields are present.

- **No transitive dependencies:**

Non-key attributes depend solely on their entity's primary key. Dependencies do not pass through intermediate attributes, preserving structural clarity.

- **Clear entity separation:**

Individual entities such as **Customer**, **Address**, **Orders**, **MenuItem**, **Cook**, and **Delivery** are distinctly defined, preventing overlap and ensuring precise data representation.

Overall, this normalization approach results in:

- Reduced redundancy
- Improved data integrity
- Easier maintenance
- Consistent and scalable database development

11. Practical Scenario Walkthrough

The following steps illustrate a real-world workflow in the FoodO system, based on the relationships shown in the ER diagram:

- **User registration:**

A new user is created in the system, storing essential information such as name and email. This User is linked to a corresponding Customer profile.

- **Customer places an order:**
The Customer entity initiates an order, identified by attributes like **orderID** and **kitchen**.
- **Order contains menu items:**
The order is associated with multiple MenuItem s through the **Contains** relationship. Each MenuItem includes attributes such as **name**, **description**, and **price**.
- **Food preparation:**
The order is assigned to a **Cook**, who is responsible for preparing the items in that order.
- **Delivery process:**
The completed order is linked to a specific **Address** through the **Delivers** relationship. Delivery assignment details are managed using the **Assigns** relationship.
- **Customer feedback:**
After receiving the order, the customer may provide a **Review** for the MenuItem, contributing feedback to the system.

4.3 Process Flow/Representation

The process flow of foodo represents how different user types—Customers, Cooks, Riders, and Admin—interact with the platform to complete essential actions such as browsing meals, placing orders, preparing food, delivering items, and managing the system. The flow demonstrates the step-by-step sequence of activities across the platform, ensuring smooth coordination between all participants.

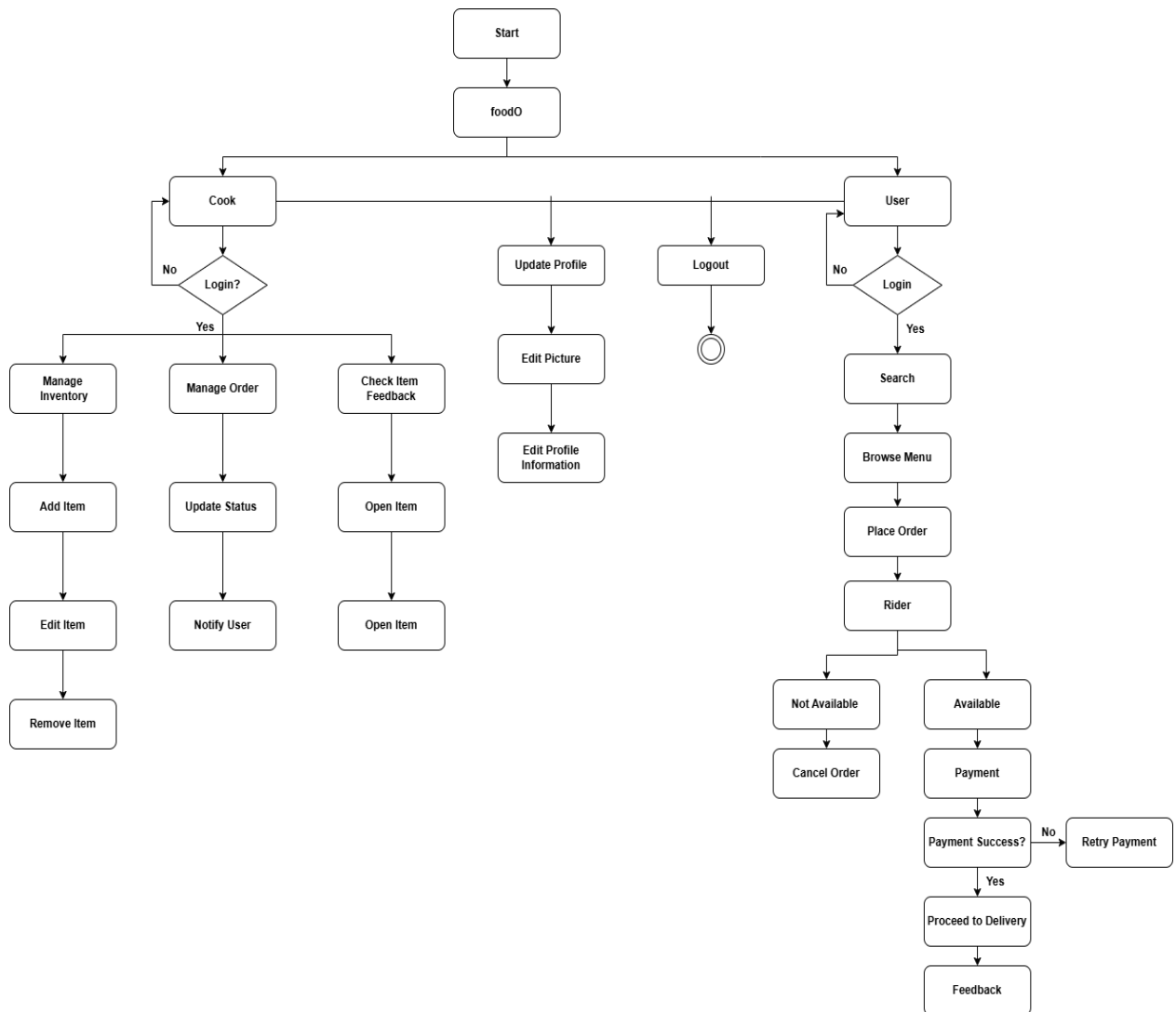


Figure 4. 3 Process flow

1. Start of the System

The system begins at the **Start** node, where users choose their role upon opening foodo. The platform supports four user types—Customer, Cook, Rider, and Admin—each with a distinct workflow. Depending on the selected role, the system directs the user to the appropriate authentication process. If login credentials are incorrect, the system displays an error and prompts for re-entry, ensuring only authorized access.

2. Customer Flow

Authentication

- Customer registers or logs in using valid credentials.

Browsing & Selection

- Browses food items and menus created by cooks.

- Searches meals, views item details, and manages profile settings.

Cart & Checkout

- Adds selected items to cart.
- Proceeds to checkout and initiates payment.

Payment Processing

- System verifies payment:
 - **Success:** Order is placed.
 - **Failure:** User is prompted to retry or choose another method.

Order Tracking

Customer tracks order status in real-time:

- Accepted → Preparing → Ready for Pickup → Out for Delivery → Delivered.

Feedback

- After delivery, customer may leave a rating or review.

Logout

- Customer logs out after completing actions.

3. Cook Workflow

Authentication

- Cook logs into the platform.

Menu Management

- Adds new food items.
- Edits existing item details, prices, descriptions.
- Disables unavailable items.

Order Handling

- Views incoming customer orders.
- Updates order statuses:
 - Accepted
 - Preparing
 - Ready for Pickup

Customer Feedback

- Views customer reviews to monitor quality.

Logout

- Cook logs out once tasks are completed.

4. Rider Workflow

Authentication

- Rider logs into the system.

Order Assignment

- Receives list of available delivery tasks.
- Accepts an order once cook marks it “Ready for Pickup.”

Pickup & Navigation

- Picks up the prepared order from the cook.
- Uses geolocation API for optimized navigation.

Status Updates

- Updates stages:
 - Picked Up
 - On the Way
 - Delivered

Completion

- Marks delivery as completed, updating customer and system records.

Logout

- Rider logs out after completing deliveries.

5. Admin Workflow

Authentication

Admin logs into the admin dashboard.

User Management

- Manages Customer, Cook, and Rider accounts.
- Approves new cook registrations.
- Handles complaints, reviews, and disputes.

System Monitoring

- Oversees platform analytics, system logs, and performance.

Content Moderation

- Reviews flagged items, feedback, or suspicious activity.

Logout

- Admin logs out after system checks and updates.

6. System End

- Each user role—Customer, Cook, Rider, Admin—ends at the **Logout** → **End** node.
- Ensures account security and session closure.

4.4 Design Models

4.4.1 Sequence Diagram

The sequence diagram for **foodo** illustrates the complete end-to-end flow—starting from a customer placing an order, the cook preparing it, the rider delivering it, and the system updating all roles in real time.

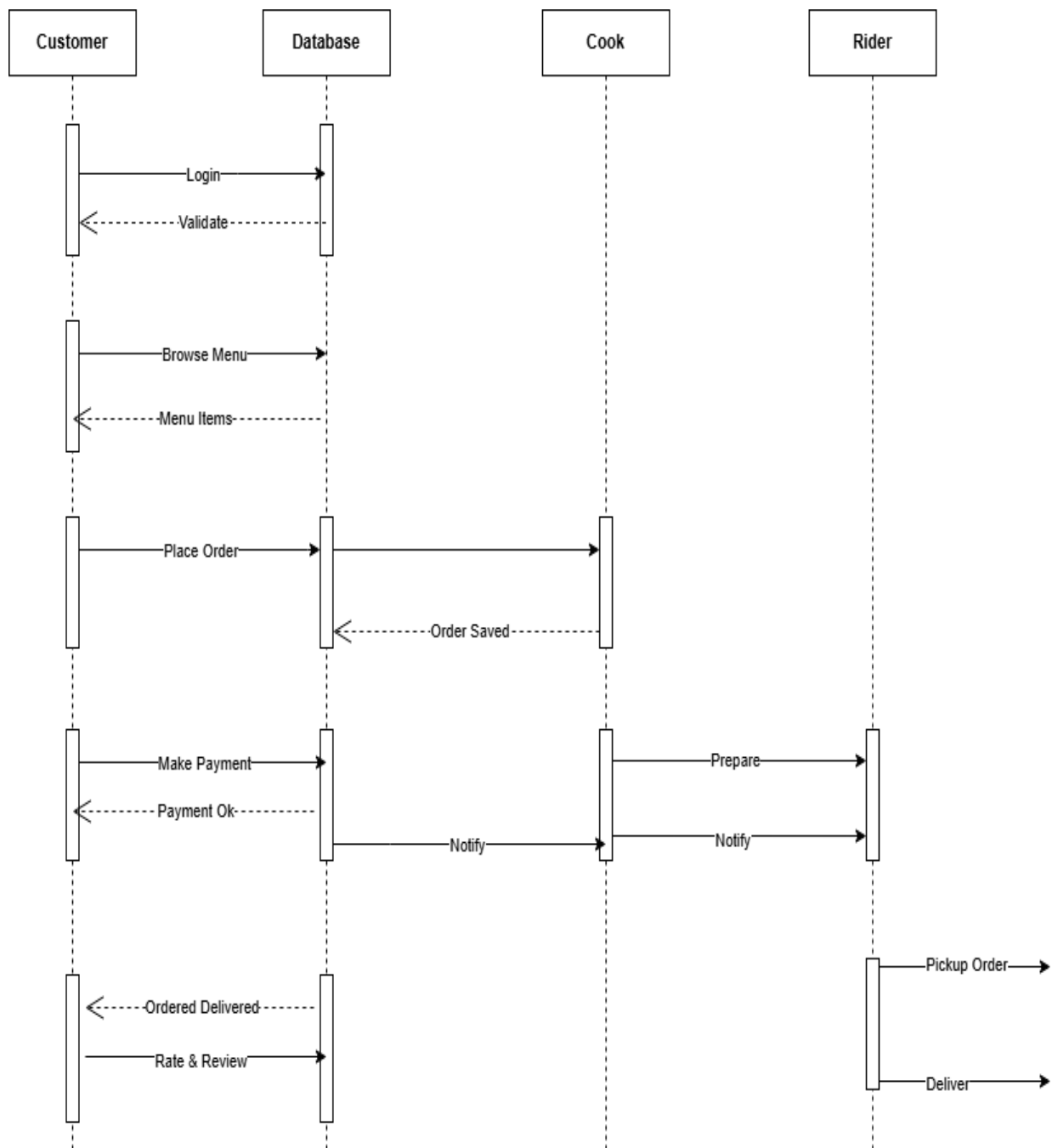


Figure 4. 4 Sequence Diagram

1. Participants and Their Roles

a) Customer

The end-user who browses meal items, places an order, makes a payment, and tracks delivery status.

b) foodO System

The core application server that processes requests, manages business logic, assigns tasks, and synchronizes updates between all user roles.

c) Database (MongoDB)

Stores all persistent information including user accounts, menu items, orders, payment details, rider locations, and delivery statuses.

d) Cook

A home-based food provider who receives customer orders, updates order preparation stages, and marks orders ready for pickup.

e) Rider

Responsible for picking up prepared orders, delivering them to the customer, updating live delivery stages through the system.

2. Sequence of Operations (Step-by-Step Explanation)

Step 1: User Authentication

- **Customer → foodO System:** Login / Registration request
- **System → Database:** Verify user credentials
- **Database → System:** Authentication success
- **System → Customer:** Access granted

This ensures only verified users access the platform.

Step 2: Browsing Menu Items

- **Customer → System:** View menu / Search meal
- **System → Database:** Fetch meal items listed by cooks
- **Database → System:** Return menu details (images, prices, descriptions)
- **System → Customer:** Display available meals

Customers can browse cooks, categories, and daily special meals.

Step 3: Adding Items to Cart

This step repeats until the customer finalizes their selection.

- **Customer → System:** Add item to cart
- **System → Customer:** Confirm item added

A loop is maintained to allow multiple items before checkout.

Step 4: Checkout & Payment

- **Customer → System:** Checkout
- **System → Customer:** Payment options (COD, PayPal)
- **Customer → System:** Make payment
- **System → Database:** Store payment and order record
- **Database → System:** Stored successfully
- **System → Cook:** New Order Notification
- **Cook → System:** Order Accepted

The order officially enters the preparation stage after payment.

Step 5: Order Preparation (Cook Side)

- **Cook → System:** Update status → *Preparing*
- **System → Customer:** Real-time update
- **Cook → System:** Update status → *Ready for Pickup*
- **System → Rider:** New Delivery Request

Real-time updates use **Socket.io** events.

Step 6: Rider Pickup & Delivery

- **Rider → System:** Accept delivery
- **System → Cook:** Rider assigned
- **Rider → Cook:** Pickup food
- **Rider → System:** Update status → Picked Up
- **System → Customer:** Live tracking enabled
- Rider location is regularly updated using the **Geolocation API**.
- **Rider → System:** Update status → On the Way
- **System → Customer:** Show map tracking
- **Rider → System:** Update status → Delivered
- **System → Customer:** Order Delivered message

This completes the order lifecycle.

Step 7: Feedback Submission

- **Customer → System:** Submit rating/review
- **System → Database:** Save feedback
- **System → Cook:** New review notification

Feedback supports quality improvement for cooks.

3. Key Features of the Sequence Diagram

a) Real-Time Events

- Socket.io enables:
- Live status updates
- Order notifications
- Rider location tracking

b) Role-Based Interactions

- Each actor performs only their allowed functions:
- Customer → ordering and payment
- Cook → preparation updates
- Rider → pickup and delivery
- System → coordination
- Database → storage

c) Loop for Cart Items

Allows adding multiple items dynamically.

d) Sequential, Event-Driven Logic

Each step depends on the completion of the previous one, ensuring consistent workflow.

4. Importance in foodO System Design

- The sequence diagram helps in:
- Understanding communication between system modules
- Verifying logical correctness of the order workflow
- Supporting developers during implementation
- Assisting testers in preparing test cases
- Documenting how real-time delivery systems behave

5. Summary of Overall User Flow

- User logs in
- Browses meals and adds items to cart
- Performs payment (COD/PayPal)
- Cook receives and prepares the order
- Rider picks up and delivers
- Customer tracks status live
- Customer leaves a review

4.4.2 Class Diagram

Displays system components such as **User**, **Cook**, **Rider**, **MenuItem**, **Order**, **Review**, **Payment**, and their relationships. Attributes and methods are defined to show structure and behavior.

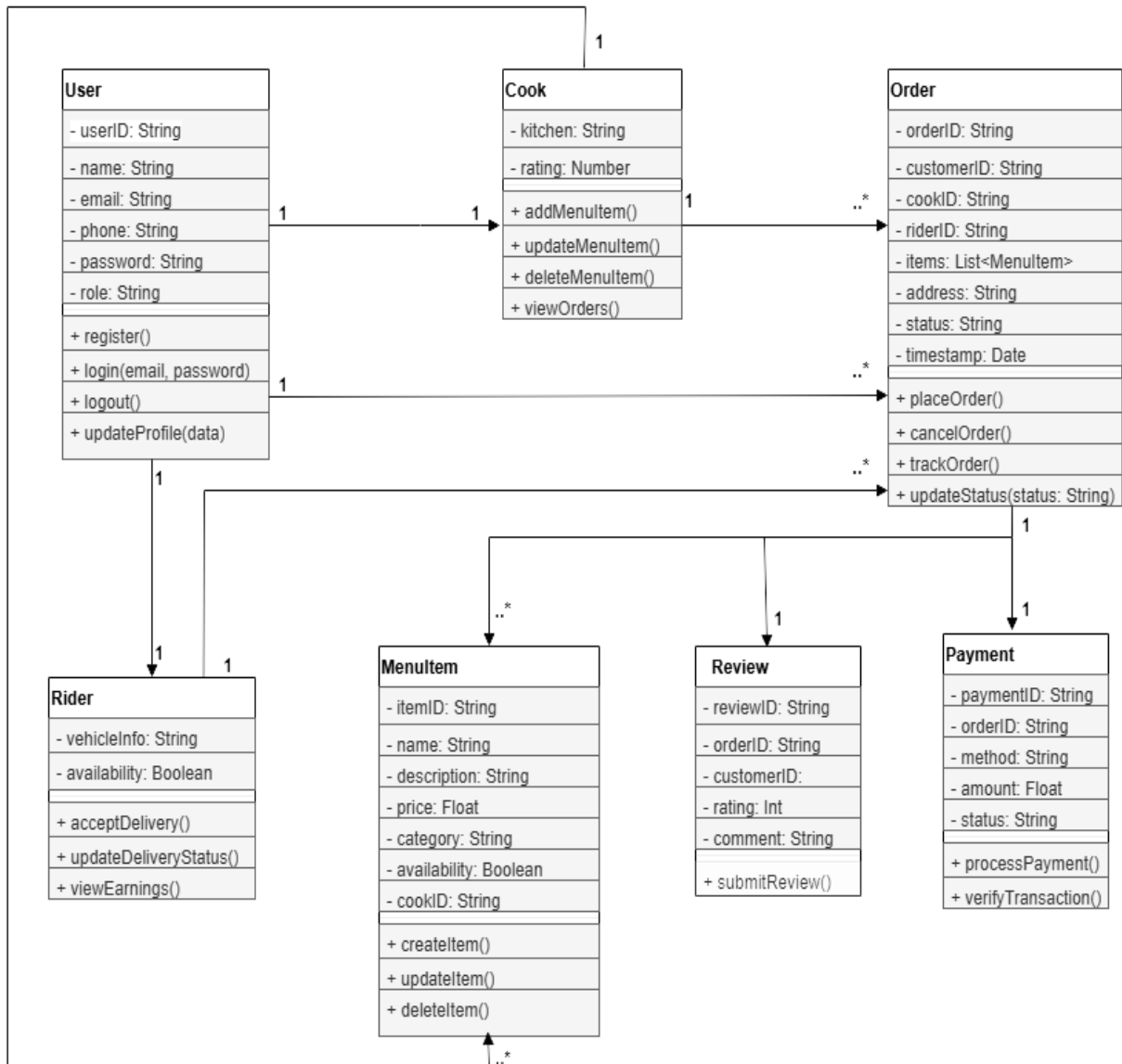


Figure 4. 5 Class Diagram

1. User Class

a) Attributes:

- **userID**
Unique identifier for each user.
- **name**
Full name of the user.

- **email**
Email used for registration and communication.
- **phone**
Contact number of the user.
- **password**
Encrypted password for secure login.
- **role**
Specifies whether the user is a Customer, Cook, or Rider.

b) Methods:

- **register()**
Creates a new account in the system.
- **login(email, password)**
Authenticates the user and grants system access.
- **logout()**
Ends the active session.
- **updateProfile(data)**
Updates user information such as name, phone, or profile details.

c) Relationships:

- A **User** can place many **Orders** (1 to many).
- A **User** can give many **Reviews** (1 to many).
- A **User** may be linked to **Cook** or **Rider** roles based on their assigned role.

2. Cook Class

a) Attributes:

- **kitchen**
Represents the kitchen name or cooking workspace of the cook.
- **Rating**
Overall rating based on customer feedback.

b) Methods:

- **addMenuItem()**
Allows the cook to create a new menu item.
- **updateMenuItem()**
Edits details of an existing menu item.

- **deleteMenuItem()**
Removes a menu item from the cook's listing.
- **viewOrders()**
Shows orders received from customers.

c) Relationships:

- One **Cook** has many **MenuItem** entries (1 to many).
- One **Cook** receives many **Orders** (1 to many).

3. Rider Class

a) Attributes:

- **vehicleInfo**
Details about the rider's vehicle used for deliveries.
- **availability**
Boolean value indicating whether the rider is available for delivery.

b) Methods:

- **acceptDelivery()**
Allows riders to accept order delivery requests.
- **updateDeliveryStatus()**
Updates the delivery progress such as "Picked Up" or "Delivered".
- **viewEarnings()**
Displays total earnings accumulated from completed deliveries.

c) Relationships:

- One **Rider** delivers many **Orders** (1 to many).

4. MenuItem Class

a) Attributes:

- **itemID**
Unique identifier for each menu item.
- **name**
Name of the food item.
- **description**
Detailed explanation of the dish.
- **price**
Cost of the item.

- **category**
Category such as snacks, meals, desserts, etc.
- **availability**
Boolean to indicate whether the item is currently available.
- **cookID**
Links the item to the cook who created it.

b) Methods:

- **createItem()**
Adds a new dish to the cook's menu.
- **updateItem()**
Modifies an existing dish.
- **deleteItem()**
Removes the dish from the list.

c) Relationships:

- A **MenuItem** belongs to one **Cook** (many to one).
- A **MenuItem** can receive many **Reviews** (1 to many).
- A **MenuItem** can be part of many **Orders** (many to many through items list).

5. Order Class

a) Attributes:

- **orderID**
Unique identifier for each order.
- **customerID**
User ID of the ordering customer.
- **cookID**
Identifies the cook preparing the order.
- **riderID**
Links the order to the assigned delivery rider.
- **items**
A list of MenuItems included in the order.
- **address**
Delivery address provided by the customer.
- **status**
Indicates order stage such as "Pending", "Preparing", "Out for Delivery", or "Delivered".

- **timestamp**
Date and time of order placement.

b) Methods:

- **placeOrder()**
Creates and submits a new order.
- **cancelOrder()**
Cancels the order before preparation or dispatch.
- **trackOrder()**
Allows customers to view real-time order status.
- **updateStatus(status)**
Lets cooks or riders update order progress.

c) Relationships:

- One **Order** belongs to one **Customer**.
- One **Order** is prepared by one **Cook**.
- One **Order** is delivered by one **Rider**.
- One **Order** contains multiple **MenuItems**.
- Each **Order** is linked to exactly one **Payment**.

6. Review Class

a) Attributes:

- **reviewID**
Unique identifier for the review.
- **orderID**
Identifies the order being reviewed.
- **customerID**
User ID of the reviewing customer.
- **rating**
Numeric rating score.
- **comment**
Written feedback provided by the customer.

b) Methods:

- **submitReview()**
Allows users to submit their rating and comments.

c) Relationships:

- A **Review** is created by one **User**.
- A **Review** belongs to one **Order**.
- A **MenuItem** may have many **Reviews**.

7. Payment Class

a) Attributes:

- **paymentID**
Unique identifier for the payment transaction.
- **orderID**
Links payment to a specific order.
- **method**
Payment method used (e.g., PayPal, COD).
- **amount**
Final amount charged.
- **status**
Indicates whether the payment is “Completed”, “Pending”, or “Failed”.

b) Methods:

- **processPayment()**
Initiates and processes the payment.
- **verifyTransaction()**
Validates that the payment was successful and secure.

c) Relationships:

- One **Payment** is associated with one **Order** (1 to 1).

8. Entity Relationships Summary

Here’s a quick overview of cardinalities:

Table 4. 3 Class Diagram Relationship

Relationship	Cardinality

User → Order	1 to many
User → Review	1 to many
Cook → MenuItem	1 to many
Cook → Order	1 to many
Rider → Order	1 to many
MenuItem → Review	1 to many
Order → MenuItem	1 to many (via list)
Order → Payment	1 to 1

9. Use Cases and Real-World Mapping

- A customer logs in, browses menu items created by cooks, and places an order.
- The order is stored with links to the correct cook and rider.
- Payment is processed and stored in the Payment class.
- After delivery, the customer submits a review linked to the order and menu items.
- Cooks continuously update menu items and view incoming orders.
- Riders accept delivery tasks, update delivery statuses, and track earnings.

This structure ensures clear relationships, strong data integrity, and smooth system scalability.

Chapter 5

Implementation

5 Implementation

The implementation of the foodO system involved building a full-stack web application designed for customers, home cooks, and riders. This chapter explains how each module was developed and integrated into a complete working system. The system was built using the Incremental Software Development Model, allowing core functionalities to be implemented first and additional modules added over time. This iterative approach ensured flexibility during development, enabling adjustments based on testing and user feedback.

The entire foodO platform is developed using modern JavaScript technologies:

- **Backend**

Node.js with Express.js and MongoDB (Mongoose) to create RESTful APIs.

- **Frontend (Web Application)**

ReactJS for a responsive and interactive user interface for customers, cooks, riders, and admins.

- **Real-Time Module**

Socket.IO for instant communication during order delivery.

5.1 Backend Implementation

The backend serves as the core logic of the foodO system. It manages authentication, user roles, menu items, order processing, and rider delivery workflows.

Key Implementation Details

- **Authentication**

JSON Web Tokens (JWT) are used for secure login and session handling for customers, cooks, and riders. Passwords are encrypted using bcrypt before being stored in the database.

- **Routing & Controllers**

A modular folder structure separates routes, controllers, models, and middleware. This approach allows clean code management and simplified debugging.

- **Database Modeling**

MongoDB is used to store dynamic data. Major collections include Users, MenuItems, Orders, Reviews, Riders, and Payments.

- **Image Handling**

Images for food items or cook profiles are uploaded to the server and stored with unique IDs to prevent conflicts.

- **Real-Time Delivery Updates**

Socket.IO enables live communication for delivery tracking, including rider acceptance, pickup, and delivery completion.

5.2 Frontend Implementation (ReactJS)

The frontend of foodO is developed using **ReactJS**, providing a responsive and seamless interface for all user roles. It includes role-based views so customers, cooks, riders, and admins only access the features relevant to them.

Key Components

- **Authentication Screens**

Pages for registration and login with form validation and API-based authentication.

- **Customer Interface**

Includes restaurant/cook browsing, menu viewing, item details, cart operations, checkout, and order tracking.

- **Cook Dashboard**

Allows home cooks to add, edit, delete menu items, manage availability, and view incoming orders in real-time.

- **Rider Dashboard**

Displays assigned deliveries, allows riders to update delivery status, and shows daily earnings.

- **Admin Panel**

Provides tools for managing system users, verifying cooks, overseeing rider activity, and resolving disputes.

5.3 Database Implementation (MongoDB)

MongoDB serves as the database for foodO because of its flexibility in handling unstructured and semi-structured data. It fits well with dynamic menus, user roles, and variable order structures.

Key Database Features

- **Collections Include**

Users, Cooks, Riders, MenuItems, Orders, Reviews, Payments, and Withdrawals.

- **Schema Design**

Each schema aligns with the class diagram and maintains consistency through referenced ObjectIDs.

- **Data Relationships**

The database supports linking between customers, cooks, riders, and menu items, enabling smooth order creation and tracking.

- **Indexing**

Frequently searched fields, such as cookID or category, are indexed to enhance query performance.

5.4 API Integration

foodO integrates multiple third-party APIs to improve system functionality.

- **PayPal Integration**

Used for secure online payments. After a successful transaction, the system verifies payment status and links it to an order.

- **Stripe Integration**

Provides an additional payment option for customers preferring card payments.

- **Google Maps API**

Used for location validation, distance estimation, and mapping rider routes during delivery.

These integrations strengthen the payment process, improve order tracking, and help riders navigate efficiently.

5.5 User Interface

The foodO UI is designed for clarity, simplicity, and responsiveness. It supports multiple screen sizes and is usable on desktops, tablets, and mobile browsers.

5.5.1 Web Interface (ReactJS)

- Home Page

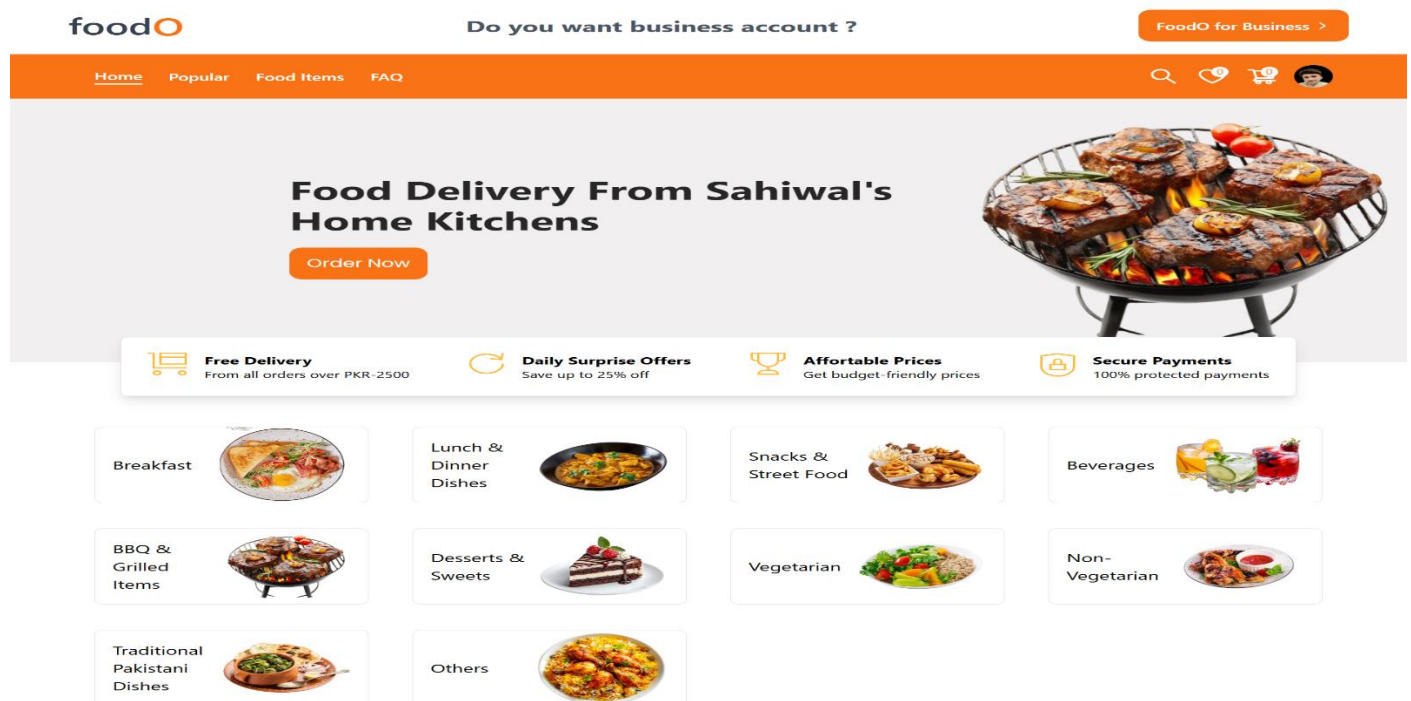


Figure 5.1 Home Page

The FoodO home page provides a clean and welcoming user interface showcasing featured deals, food categories, and promotional offers. It highlights key services such as free delivery, daily discounts, secure payments, and affordable prices. The design helps users quickly explore different food options and start ordering from home kitchens in Sahiwal.

- Menu Page

Best Deals in Sahiwal

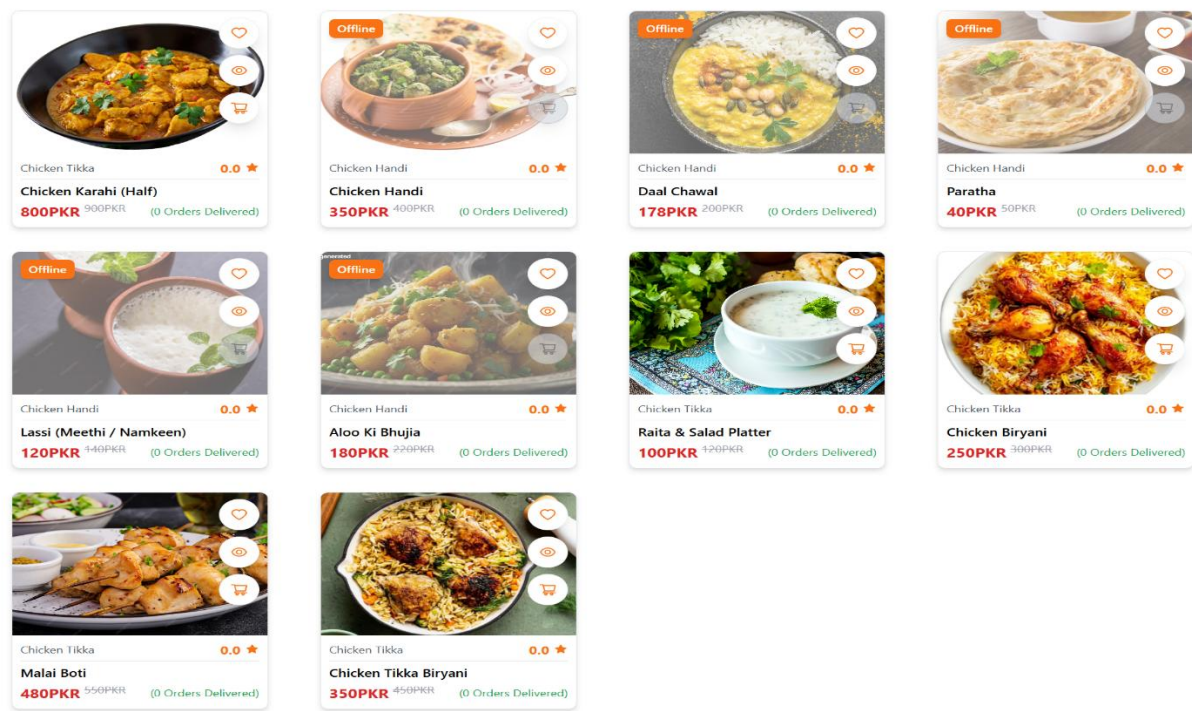


Figure 5.2 Menu Items

This page displays the Best Deals available in Sahiwal, featuring a variety of menu items with clear pricing, discounts, and delivery status. Each item card includes an appealing food image, price comparison, rating, and quick-action icons, making it easy for users to explore dishes and add favorites or cart items instantly.

- **Cart Page**

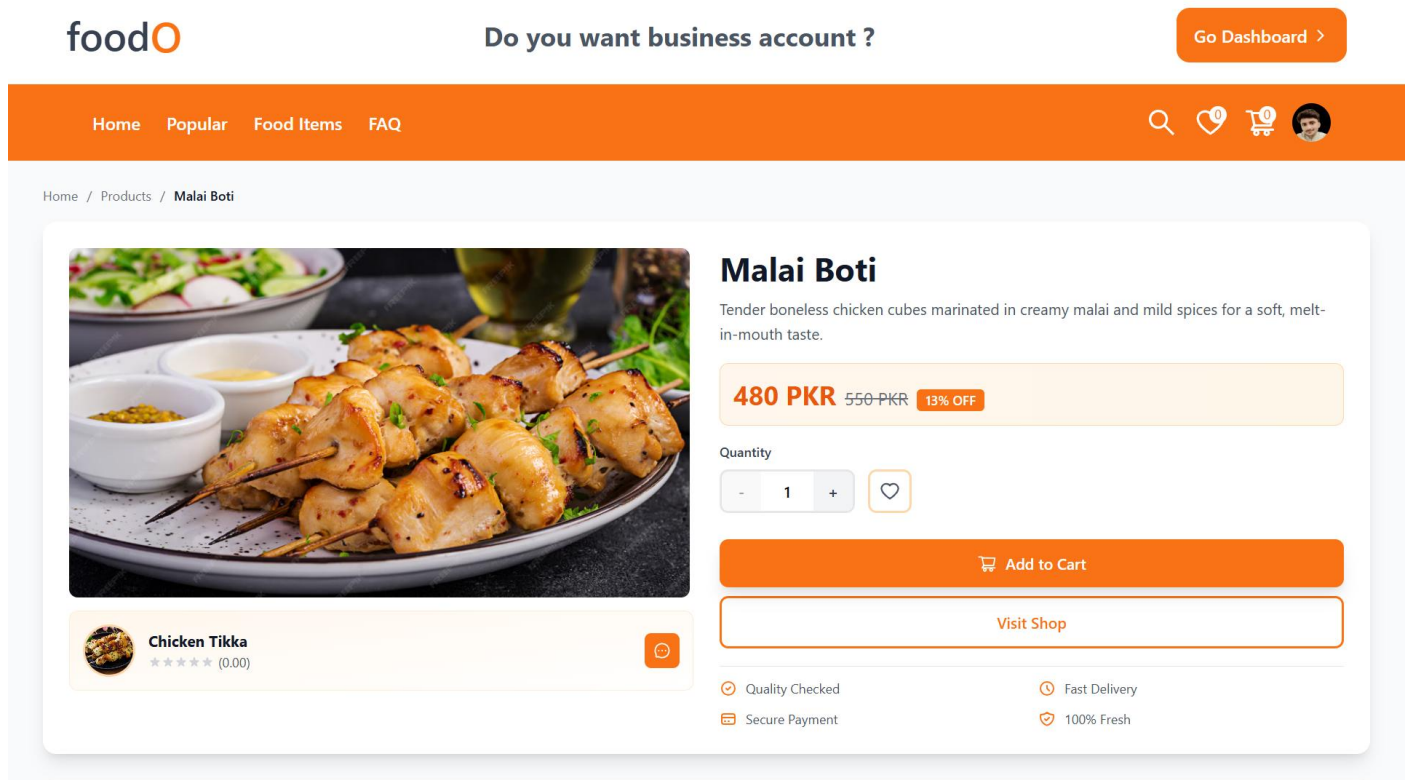


Figure 5.3 Cart Item

The Product Detail Page provides complete information about a selected dish, including high-resolution images, pricing, discount percentage, and quantity selector. Additional features such as “Add to Cart,” “Visit Shop,” quality checks, and secure payment icons enhance user confidence and streamline the ordering process.

- Cook Interface

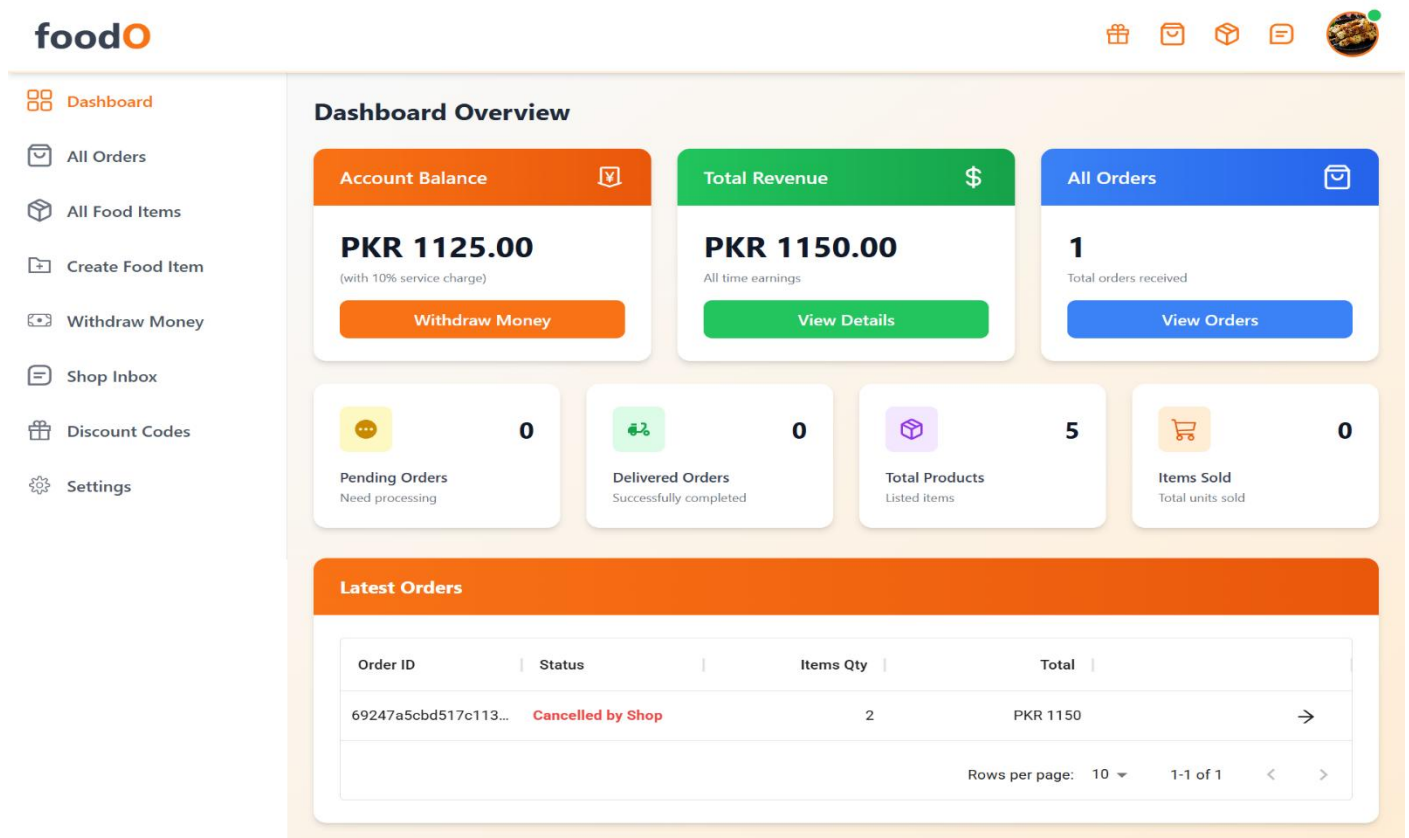


Figure 5.4 Cook Interface

The foodO Cook Dashboard provides vendors with a clear and organized overview of their business activity. It displays key metrics such as account balance, total revenue, total orders, pending orders, delivered orders, and the number of listed products. The interface also highlights the latest orders with details like order status, item quantity, and total amount. Quick navigation options on the left menu allow shop owners to manage orders, food items, withdrawals, discount codes, and settings efficiently, ensuring smooth daily operations.

- **Rider Interface**

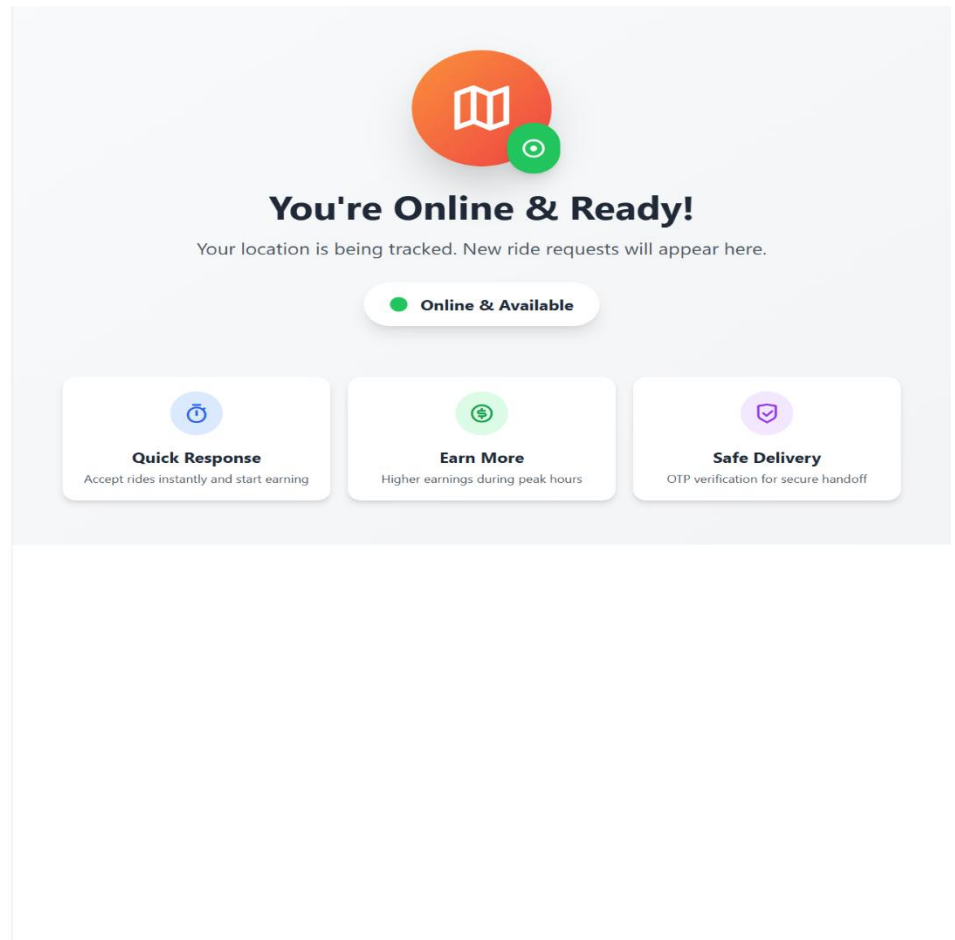
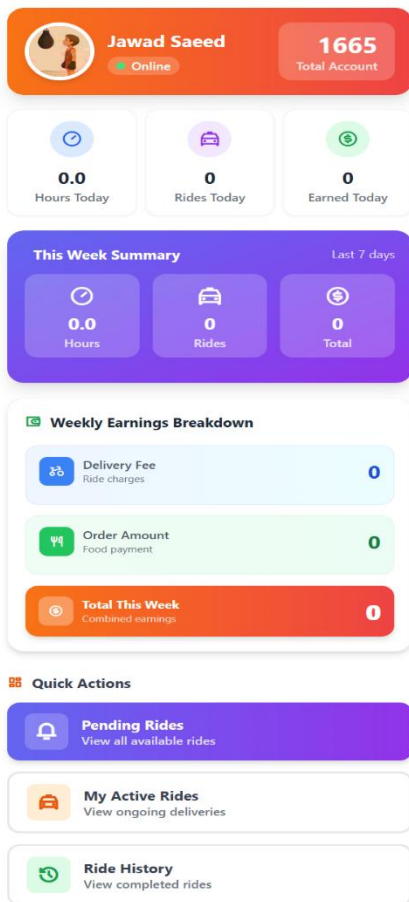


Figure 5.5 Rider Interface

The FoodO Rider Dashboard is designed to help delivery personnel efficiently manage their ride activity and earnings. It shows real-time rider status, weekly earnings breakdown, completed rides, and daily statistics. The interface includes quick access to pending rides, active rides, and ride history. With features such as “Online & Available” status, quick response options, and safe delivery verification, the dashboard ensures that riders can handle delivery tasks effectively while tracking their performance and income.

Chapter 6

API References

6 API Reference

6.1 Authentication

Table 6.1 Customer Authentication Routes

Endpoint	Method	Description	Parameters
/auth/register	POST	Register customer	name, email, password, phone, address
/auth/login	POST	Login customer	email, password

Table 6.2 Cook & Rider Authentication Routes

Endpoint	Method	Description	Parameters
/auth/register/cook	POST	Register cook	name, email, password, phone, kitchenName, address
/auth/login/cook	POST	Login cook	email, password
/auth/register/rider	POST	Register rider	name, email, password, phone, vehicleType
/auth/login/rider	POST	Login rider	email, password

6.2 Profile Management

Table 6.3 Customer Profile Management Routes

Endpoint	Method	Description	Parameters
/customer/:id/profile/image	PUT	Upload customer profile picture	image (file upload)
/customer/:id/update	PUT	Update customer profile	name, phone, address

/images/:imageId	GET	Fetch uploaded image	-
------------------	-----	----------------------	---

Table 6. 4 Cook & Rider Profile Management Routes

Endpoint	Method	Description	Parameters
/cook/:id/profile/image	PUT	Upload cook profile/kitchen image	image (file upload)
/cook/:id/update	PUT	Update cook profile	name, kitchenName, phone, address
/rider/:id/profile/image	PUT	Upload rider profile picture	image (file upload)
/rider/:id/update	PUT	Update rider profile	name, phone, vehicleType

6.3 Menu Items Management

Table 6. 5 Menu Items (Cook)

Endpoint	Method	Description	Parameters
/cook/:id/menuitems	POST	Add new menu item	name, description, price, category, stock, image
/cook/:id/items	GET	Get all menu items	—
/cook/:cookId/item/:itemId	GET	Get single item	—
/cook/:cookId/item/:itemId	DELETE	Delete item	—
/cook/:cookId/item/:itemId	PATCH	Update menu item	all item fields + image

6.4 Cook Discovery

Table 6.6 Cook Discovery (Customer)

Endpoint	Method	Description	Parameters
/cooks	GET	List all available cooks	—
/cooks/batch	POST	Fetch cooks by IDs	cookIds[]
/cook/items/:itemId	GET	Get single cook menu item	—
/cook/items/batch	POST	Get multiple menu items by ID	itemIds[]

6.5 Orders & Checkout

Table 6.7 Customer Orders & Checkout

Endpoint	Method	Description	Parameters
/checkout	POST	Place a new order	userId, cookId, name, phone, address, cart, notes
/customer/:userId/orders/active	GET	Get active customer orders	—
/customer/:userId/orders/history	GET	Get historical customer orders	—
/customer/:userId/order/:orderId	GET	Get single order	—
/customer/:userId/order/history/:orderId	GET	Get single historical order	—

Table 6.8 Cook & Rider Order Management

Endpoint	Method	Description	Parameters
/cook/:cookId/orders/active	GET	Get active cook orders	—
/cook/:cookId/orders/history	GET	Get cook order history	—
/cook/:cookId/order/:orderId	GET	Get order details	—
/rider/:riderId/orders/assigned	GET	Get rider's assigned orders	—

/rider/:riderId/order/:orderId/status	PUT	Update delivery status	status
---------------------------------------	-----	------------------------	--------

6.6 Reviews & Ratings

Table 6.9 Reviews & Ratings

Endpoint	Method	Description	Parameters
/cook/item/:itemId/reviews/:userId	POST	Submit review for menu item	rating, comment
/cook/item/:itemId/reviews	GET	Get all reviews for item	—

6.7 Media Handling

Table 6.10 Media Handling

Endpoint	Method	Description	Parameters
/images/:imageId	GET	Fetch uploaded image	-

6.8 Real-Time Features (Socket.IO)

Table 6.11 Namespaces

Namespace	Path	Description
Customer	/customer	Live updates for customer order status
Cook	/cook	Real-time notifications for new orders
Rider	/rider	Live delivery updates and tasks

6.9 Customer Events

Table 6.12 Customer Events

Event	Direction	Description	Parameters
join-customer-room	Emit	Join customer private room	userId
order-created	Receive	Order placed confirmation	full order object
order-updated	Receive	Order status update	updated order

order-delivered	Receive	Order delivered notification	orderId
cancel-order	Emit	Customer requests cancellation	orderId, reason

6.10 Cook & Rider Events

Table 6. 13 Cook & Rider Events

Event	Direction	Description	Parameters
join-cook-room	Emit	Join cook real-time room	cookId
order-created	Receive	New order received	order object
order-updated	Receive	Customer or rider updated order	updated order
update-order-status	Emit	Cook updates preparation status	orderId, status
join-rider-room	Emit	Join rider delivery room	riderId
delivery-assigned	Receive	Rider gets new delivery task	order object
update-delivery-status	Emit	Rider updates delivery state	orderId, status

Chapter 7

Testing and Evaluation

7 Testing and Evaluation

This chapter presents the testing processes carried out during the development of the foodO platform. Both manual and automated testing were performed to ensure that all modules—customer, cook, rider, and admin—functioned correctly and met the system requirements.

7.1 Manual Testing

Manual testing was performed throughout the development lifecycle to validate that every feature of foodO operated according to the SRS and design specifications.

7.1.1 System testing

System testing verified the complete behavior of foodO as a fully integrated platform, ensuring that the frontend, backend APIs, database operations, and third-party services (Stripe, PayPal, Google Maps) worked smoothly.

- **Objective**

Ensure the entire foodO application functions correctly in real-world usage.

- **Scope**

Covers full workflows including:

- customer registration
- login
- browsing cooks
- viewing menu items
- placing orders
- real-time tracking
- rider delivery
- submitting reviews

- **Result**

All major end-to-end scenarios passed successfully.

7.1.2 Unit Testing

Each functional unit was tested individually to verify its correctness. Below are examples:

- **Unit Testing 1**

Login Module

- **Objective**

Ensure valid login credentials grant access, and invalid ones are rejected.

Table 7.1 Unit Test 1

No.	Test Case	Input	Expected Result	Result
1	Valid login	Email: user@foodo.com , Pass: 123456	Dashboard loads	Pass
2	Invalid password	Email: user@foodo.com , Pass: abc	Error: Incorrect password	Pass
3	Empty fields	—	Error: Fields cannot be empty	Pass

- **Unit Testing 2**

Profile Update

- **Objective**

Verify that profile information updates correctly and validation rules are enforced..

Table 7.2 Unit Test 2

No.	Test Case	Input	Expected Result	Result
1	Update phone number	New number entered	Profile updated successfully	Pass
2	Leave required field empty	Remove email	Error: Required field missing	Fail

7.1.3 Functional Testing

- Functional testing validated that each functional requirement of foodO behaved as expected.

- **Functional Testing**

Order Placement

- **Objective**

Verify that a customer can place an order successfully.

Table 7. 3 Functional Test

No.	Test Case	Input	Expected Result	Result
1	Add items and checkout	Selected 3 items, filled address	Order confirmed and ID generated	Pass
2	Attempt to place empty cart	No items selected	Error: Cart is empty	Fail

7.1.4 Integration Testing

Integration testing was conducted to ensure proper data flow between system components.

Table 7. 4 Integration Test

No.	Scenario	Input	Expected Outcome	Result
1	Login and place order	Customer logs in, selects items, places order	Order processed; cook receives order	Pass
2	Cook updates order status	Cook marks order as "Preparing" → "Ready"	Customer receives real-time updates	Pass

3	Delivery completion & review	Rider marks "Delivered", customer reviews	Review saved; item rating updated	Pass
---	------------------------------	---	-----------------------------------	------

7.2 Automated Testing:

Automated tests were executed primarily for backend endpoint validation using **Postman**. These tests checked:

- response status codes
- authentication flows
- order placement API
- menu item retrieval
- error-handling behavior

Table 7. 5 Tool Used

Tool	Description	Applied On	Results
Postman [7]	API testing tool	Register, Login, Menu APIs, Checkout API	Pass

Automated tests ensured reliable backend communication and validated that RESTful APIs returned accurate, consistent responses.

Chapter 8

Conclusion and Future Works

8 Conclusion and Future Work

Summarizes project outcomes and proposes future directions.

8.1 Conclusion

The foodO project successfully delivers a complete, web-based platform that connects customers, home-based cooks, and delivery riders within a unified digital ecosystem. By focusing on local home-cooked food services, foodO fills a major gap left by mainstream food delivery platforms that primarily cater to large restaurants. Throughout the development lifecycle, the project followed the Incremental Software Development Model, allowing continuous refinement, early testing, and smooth integration of new features.

All core modules—including customer ordering, cook menu management, rider delivery workflows, and admin supervision—were fully implemented and validated through comprehensive manual and automated testing. The system meets the defined functional and non-functional requirements of the SRS and demonstrates strong performance, scalability, and usability. With real-time updates, intuitive UI design, and a modular backend architecture, foodO achieves its goal of offering a reliable, flexible, and user-centric solution tailored for the home-cooked food delivery market.

Overall, foodO stands as a modern, efficient, and community-focused platform that enhances digital earning opportunities for local cooks, improves order transparency for customers, and provides a structured workflow for riders—successfully addressing long-standing limitations in existing delivery systems.

8.2 Future Work

While the current version of "OrderUp" is robust and fully functional, several enhancements can be made to improve system performance and user experience in future iterations:

- **Dedicated Rider Application**

Future versions may include a full-feature mobile/web app for riders, enabling more efficient navigation, delivery status handling, and earnings management.

- **Advanced Online Payments**

Integrating Stripe and PayPal for secure and seamless online transactions will offer greater flexibility and reduce reliance on cash-based payments.

- **AI-Driven Personalization**

Machine learning algorithms can be used to recommend cooks or dishes based on user ordering patterns, dietary preferences, or popular trends.

- **Push Notifications**

Adding real-time push notifications (web + mobile) can enhance communication for order confirmation, preparation updates, delivery progress, and promotional offers.

- **Multi-language Support**

Introducing Urdu and regional languages will make foodO more accessible to a wider audience across Pakistan.

- **Enhanced Real-Time Tracking**

Live GPS-based tracking for riders, improved ETA predictions, and customer alerts at each delivery stage can significantly improve the transparency and reliability of the delivery process

References

- [1] Node.js, “Node.js Documentation,” 2024. [Online]. Available: <https://nodejs.org/>
- [2] Express.js, “Express.js Guide: Web Application Framework for Node.js,” 2024. [Online]. Available: <https://expressjs.com/>
- [3] MongoDB, “MongoDB Documentation,” 2024. [Online]. Available: <https://www.mongodb.com/docs/>
- [4] Socket.IO, “Real-Time Bidirectional Event-Based Communication,” 2024. [Online]. Available: <https://socket.io/>
- [5] Z. Wang et al., “Real-time delivery tracking systems for web-based logistics platforms,” *International Journal of Transportation Science and Technology*, vol. 8, no. 4, pp. 365–376, 2019.
- [6] B. Mishra and R. Gupta, “Evaluating Node.js for scalable and high-performance web applications,” *Journal of Web Development*, vol. 15, pp. 45–52, 2019.
- [7] Postman, “API Testing & Automation Tool,” 2024. [Online]. Available: <https://www.postman.com/>
- [8] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, Addison-Wesley, 2004.
- [9] Foster, E., and T. J. Brown, *Software Engineering: A Methodical Approach*, Auerbach Publications, 2021.
- [10] FoodPanda Research Team, “Digital Transformation in Food Delivery and Home-Based Selling,” *FoodPanda Insights Report*, 2022.
- [11] Nielsen, J., “Usability Heuristics and User-Centered Interface Design,” Nielsen Norman Group, 2020. [Online]. Available: <https://www.nngroup.com/>
- [12] Stripe, “Online Payments API Documentation,” 2024. [Online]. Available: <https://stripe.com/docs>
- [13] Google Maps Platform, “Geolocation & Routing APIs,” 2024. [Online]. Available: <https://developers.google.com/maps>
- [14] PayPal, “PayPal Developer Documentation,” 2024. [Online]. Available: <https://developer.paypal.com/docs/>